Camisaedit

Proyecto de Fin de Grado

2º año de Desarrollo de Aplicaciones Web

Alumno: Iru Borredá Bin

Tutor del TFG: Antonio Jesús Carmona

Índice

ABSTRACT 5

JUSTIFICACIÓN DEL PROYECTO 6

INTRODUCCIÓN 7

ANTECEDENTES 8

Configurador Owayo 3D (de Owayo) 8

Armoria (de Azgaar) 11

OBJETIVOS 14

DESCRIPCIÓN 16

CASOS DE USO 16

CASO 1: REGISTRO 16

CASO 2: INCIO DE SESIÓN 18

CASO 3: CREACIÓN DE ROPA (Y PEDIDO) 20

CASO 4: VIEW Y EDICIÓN DE PERFIL (por implementar) 23

CASO 5: VIEW Y EDICIÓN DE PEDIDO (Edición Admin-only) 25

CASO 6: VIEW, EDICIÓN Y DELETE DE ROPA 27

DISEÑOS 29

Arquitectura de la aplicación 29

Diagrama de la base de datos. 29

Diagrama de flujo de navegación 31

Wireframe 32

DETALLES DEL PROYECTO 33

TECNOLOGÍAS 50

METODOLOGÍA 53
Metodología usada 53
Horas Estimadas (Previas a implementación) 53
Horas Estimadas (Posterior a implementación) 54
Diagrama de Gantt. 54
PRESUPUESTO 55
README y GIT 55
INSTALACIÓN Y DESPLIEGUE 56
Instalación 56
Despliegue 60
TRABAJOS FUTUROS 62
CONCLUSIONES 64
REFERENCIAS 65

ABSTRACT

Camisaedit es una web que se centra alrededor de la compra de camisetas y otras ropas del mismo tipo que el comprador puede personalizar a su gusto. Esto puede hacerlo el comprador mismo utilizando un editor de camisetas dentro de la web en sí, pudiendo ver los resultados de sus cambios en un previsualizador que se actualiza en función de las opciones que se hayan elegido en un formulario vinculado.

Camisaedit is a web that centers on allowing clients to buy t-shirts and other top clothing that they can customize to their liking. They can do this by utilizing the shirt editor that it is linked inside the website, letting them see a preview of whatever changes they have chosen to apply to the shirt by way of utilizing a form they can interact with.

JUSTIFICACIÓN DEL PROYECTO

Este proyecto está impulsado principalmente por mi deseo de poner a prueba lo que he aprendido durante el curso, desde interfaces hasta el desarrollo del backend como puede ser contactar y gestionar datos de la BBDD. Además, creo que un personalizador de camisetas puede ser un proyecto interesante debido a la cantidad de datos que se han de tratar, validar, y manejar para poder mostrarlos y interactuar con ellos de diferentes maneras.

Sí, hay varias aplicaciones web ya pre-existentes, una incluso renderiza un modelo de camiseta en tres dimensiones para permitir su visualización desde cualquier ángulo deseado (owayo), que permite la selección de todo desde patrones de camiseta hasta subir tus propias imágenes para ponerlas en la camiseta.

La aplicación servirá principalmente para el público general, es decir los clientes, quienes podrán entrar a la aplicación web para crear, editar o cancelar los pedidos de sus camisetas personalizadas a su gusto. Por supuesto, también se habilita un usuario “administrador” que tiene acceso a todos los pedidos que haya en la base de datos.

INTRODUCCIÓN

Camisaedit es una web que proporciona un servicio de personalización de productos de una manera accesible y fácil de utilizar, proporcionando funcionalidades como puede ser un previsualizador del diseño de la camiseta personalizar, o una página “perfil” que permita al usuario – tras haber iniciado sesión - ver todos los pedidos que ha hecho (también permitiendo un usuario “administrador” a entrar y gestionar todas las entradas).

Requisitos principales a cumplir ordenado según prioridades:

- Formularios funcionales y bien validados (Para las opciones de personalización, toma de datos de entrega y registro/inicio de sesión de un perfil).
- Sistema de iniciado de sesión funcional
- Interfaz accesible, atractiva y fácil de navegar.
- [Página perfil] Mostrar datos relevantes al usuario (Pedidos de camiseta hechos, opciones seleccionadas, estado de proceso) (dependiendo de roles de cliente y administrador) .
- Ser capaz de ver todos los detalles de un pedido al hacer “click” sobre una de las entradas en la página “Perfil” y editar el pedido si no está en fase de “En Producción”.

Requisitos no principales:

- El previsualizador en vivo de las opciones aplicadas a una camiseta en 2D.
- Permitir editar y borrar solo aquellas entradas que no estén en el estado “En producción”.

ANTECEDENTES

Camisaedit tiene mucha competencia dentro del negocio de camisetas personalizables, ya sea por negocios locales como en línea. En concreto a mí me ha llamado la atención uno por el nombre de Owayo.

A continuación, hablaré acerca de qué ofrece y cómo lo hace, además de un poco más de contexto en cuanto a esta empresa:

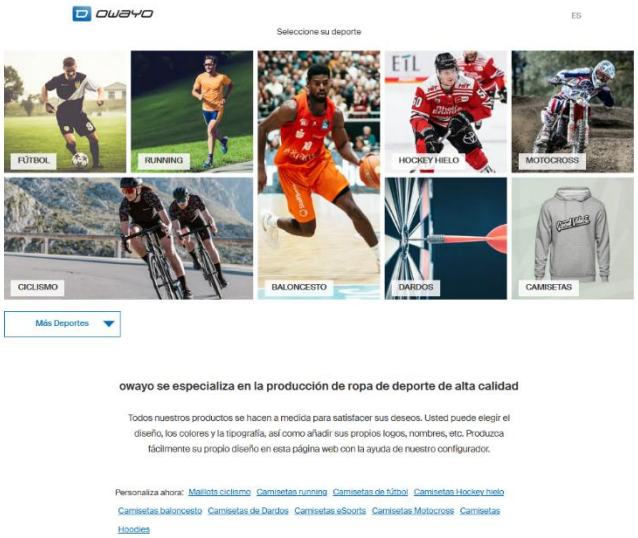
Configurador Owayo 3D (de Owayo)

Link: https://www.owayo.es/

Owayo es una empresa que se especializa en la manufacturación de camisetas deportivas de una variedad de disciplinas, reside en Alemania y hace encargos internacionales tanto en Europa como en los Estados Unidos. Asegurando máxima calidad de producto. Y su aplicación de configuración se parece mucho a lo que queremos implementar en nuestra aplicación:

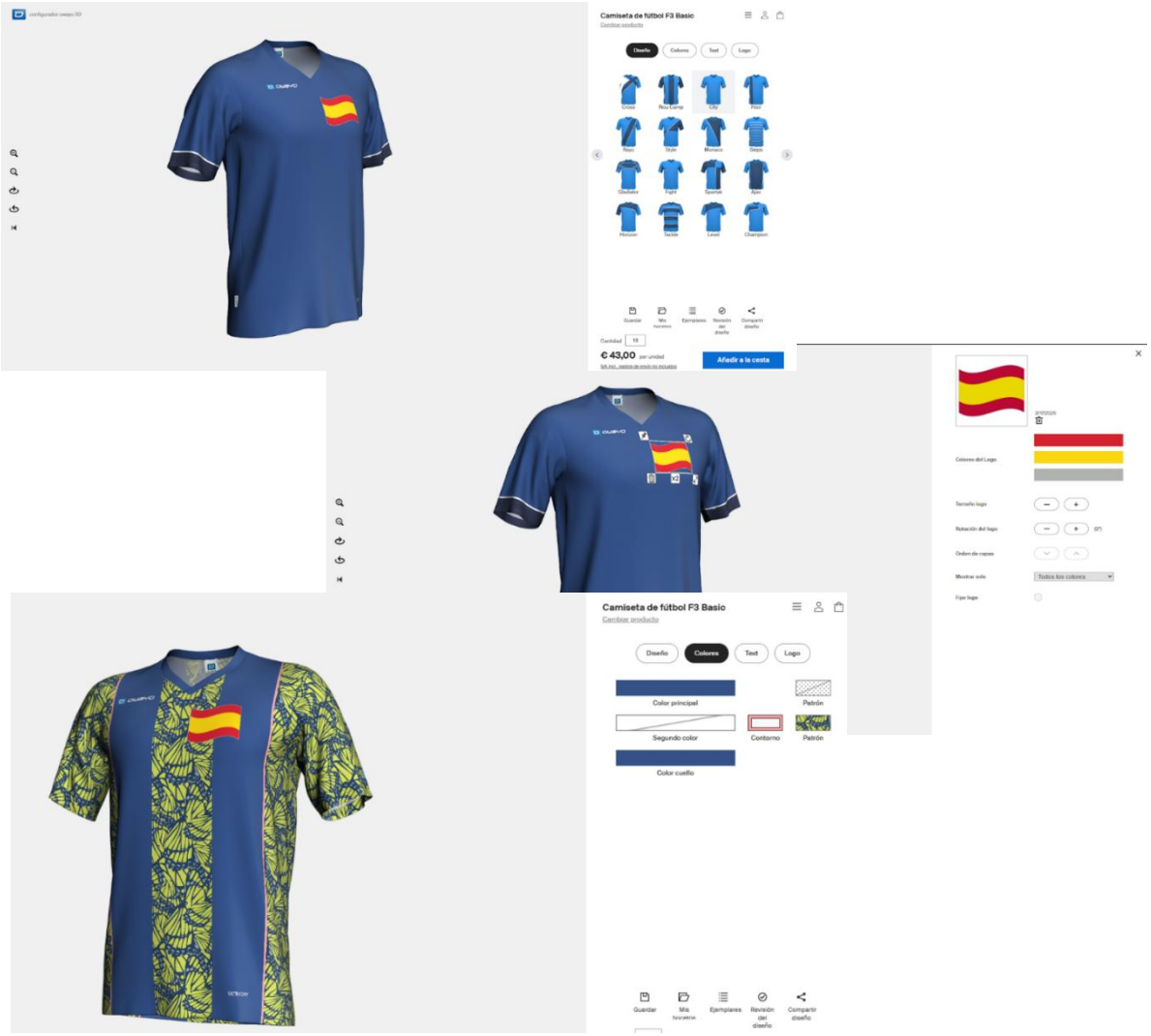
En ordenador

Al entrar en su página, vemos todos los productos que ofrecen, todos los productos de varios deportes, desde baloncesto como motocross e incluso



hockey sobre hielo.

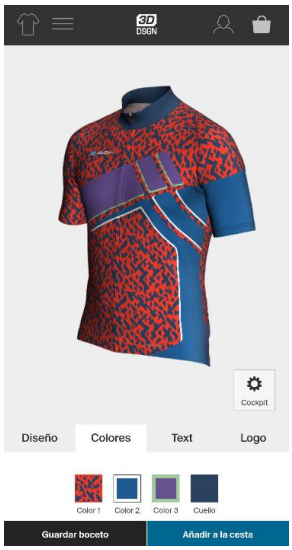
Y al entrar en su configurador de camisetas, somos recibidos por un renderizado en 3D de la ropa, con una barra lateral que nos ofrece las diferentes opciones a personalizar a nuestro gusto, como puede ser el diseño de la camiseta, los colores y los patrones de estos, si debe haber un texto en alguna parte y la posibilidad de tanto subir un logo propio como coger uno de la biblioteca que tienen para elegir, que incluye banderas, un logo que se plasma sobre la superficie de la camiseta misma y se puede posicionar libremente:



Como se puede ver, también se le muestra al cliente en todo momento el precio de la camiseta por unidad, al igual que permite editar cuantas unidades de dicha camiseta se quieren.

En Móvil

En móvil la aplicación web es perfectamente usable, permitiendo el uso de todas sus funciones sin ningún problema y con una interfaz que se adapta a la pantalla del móvil:



Conclusiones de Owayo

Esta aplicación es una de las mejores que he visto en cuanto a lo que yo mismo busco hacer, proporcionar un configurador de camisetas el cual pueda funcionar para dar imprimir y mandar lo que quieran los usuarios, y no cabe duda que está muy completo y está hecho de forma robusta.

Como se podrá asumir, yo, un programador que solo está empezando su carrera de informático, no podría competir contra el equipo de profesionales dedicados que ha tenido que programar el configurador, incluyendo la vista 3D completamente rotable. No sabría por donde empezar en cuanto a mirar su código.

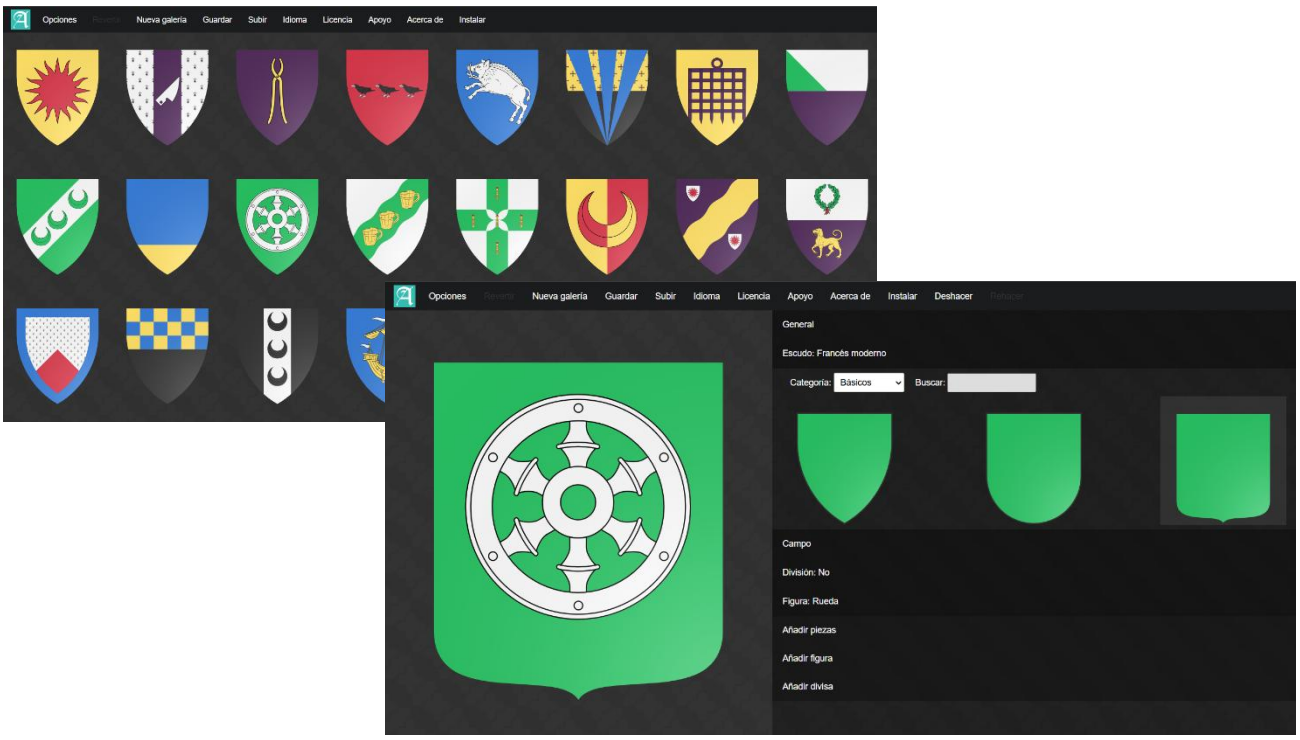
Esta aplicación sería la forma idónea de nuestro proyecto y no tiene ni punto de comparación con la nuestra.

Este, sin embargo, no ha sido el único configurador al que he estado estudiando, ya que existe otra aplicación web que utilizo a menudo para mis "hobbies" el cual me ha ayudado a ver una forma un tanto más simple de llegar a la meta que quiero: Armoria

Armoria (de Azgaar)

Link: https://azgaar.github.io/Armoria/

Armoria es una aplicación web simple hecha por Azgaar, un programar solo que es conocido por hacer herramientas para facilitar la labor de directores de juego de juegos de rol de sobremesa. En este caso, esta aplicación genera escudos medievales y fantásticos y permite editarlos como al cliente le plazca.



Dentro del editor de escudos se puede cambiar de todo, desde la forma del escudo, los símbolos dibujados sobre este al igual que patrones y demás personalizaciones, todo a través de una serie de formularios que incluso tienen vistas de galería para enseñar las opciones de manera visual.

Pero lo que vamos a coger de aquí es el propoio previsualizador, que utiliza una etiqueta svg para hacer sus formas (que actúan como una “máscara”) y aplicar los cambios que el usuario elija.

```
<svg id="coaEdit" class="coa" width="100%"
height="100%" viewBox="0 10 200 200" xmlns="htt
p://www.w3.org/2000/svg" xmlns:xlink="http://ww
w.w3.org/1999/xlink" xmlns:dc="http://purl.org/
dc/elements/1.1/" xmlns:rdf="http://www.w3.org/
1999/02/22-rdf-syntax-ns#"> == $0
  <defs>…</defs>
  <g clip-path="url(#shield_626477270)">
    <rect class="field" x="0" y="0" width="200"
    height="200" fill="#26c061"></rect>
    <g class="charge" i="0" charge="wheel"
    fill="#fafafa" stroke="#000" style="--secon
    dary: #fafafa; --tertiary: #fafafa; --strok
    e: #000; --background: block;">…</g>
  </g>
  <path class="grad" d="m 25,25 h 150 v 139.15 c
  0,41.745 -66,18.15 -75,36.3 -9,-18.15 -75,5.44
  5 -75,-36.3 v-139.15 z" fill="url(#backlight)"
  stroke="#333333" stroke-width="1" style="point
  er-events: none;"></path>
```

Debido a que un previsualizador completo se escapa del alcance del proyecto, no he investigado más acerca del proyecto, pero el meterme en el repositorio de Github de Armoria y ver que el proyecto se ha hecho en su mayoría con puro HTML, CSS y JS, me hace pensar que sería posible hacer lo mismo con mi proyecto más tarde.

https://github.com/Azgaar/Armoria

OBJETIVOS

Listado de objetivos que se plantean resolver. Requisitos.

Está representado como un **RFTP**.

R – Requisitos: Lo que debe hacer el programa expresado en lenguaje coloquial.

F – Funciones: Desglose de las características asociadas o subrequisitos de cada requisito. Expresado en leguaje técnico.

T – Tareas asociadas a cada funcionalidad. Deben describir completamente su alcance.

P – Pruebas. Demostración o prueba planificada para cumplir cada tarea.

- R01 – El programa ha de tener una estructura HTML base, acompañado de sus reglas CSS y scripts PHP y JS respectivos.
- R02 – El programa ha de poder permitir acceso a una página “perfil” a personas que se hallan creado un usuario.
    - R02F01 – El Usuario debe poder registrarse en el sistema.
        - R02F01T01 – Crear una tabla de usuarios en la base de datos
            - R02F01T01P01 – Introducir datos de prueba.
    - R02F02 – El Usuario debe poder ver datos de su cuenta y una lista de pedidos que haya realizado.
        - R02F02T01 – Crear una tabla de pedidos en la base de datos vinculada a la tabla de usuarios.
            - R02F02T01P01 – Entrar al usuario y crear pedidos a través de su cuenta.
        - R02F02T02 – El Usuario ha de ser capaz de borrar uno de los pedidos visualizados con un click y confirmación.

        - R02F02T02P01 – Entrar al usuario y borrar una de las entradas mostradas en la página “Perfil”
    - R02F03T03 – El usuario ha de ser capaz de hacer “Click” sobre una de las opciones para poder visualizar toda la información del pedido, y ser capaz de editar cualquier detalle de este.
- R03 – El programa ha de tener un formulario de colección de opciones elegidas para la camiseta
    - R03F01 – El usuario va a poder elegir de entre muchas opciones, yendo desde el tamaño de la camiseta, si tendría un logo, texto, color del texto… Etc.
        - R03F01T01 – Crear un formulario que lo permita
            - R03F01T01P01 – Entrar a dicho formulario y crear una nueva camiseta y pedido.

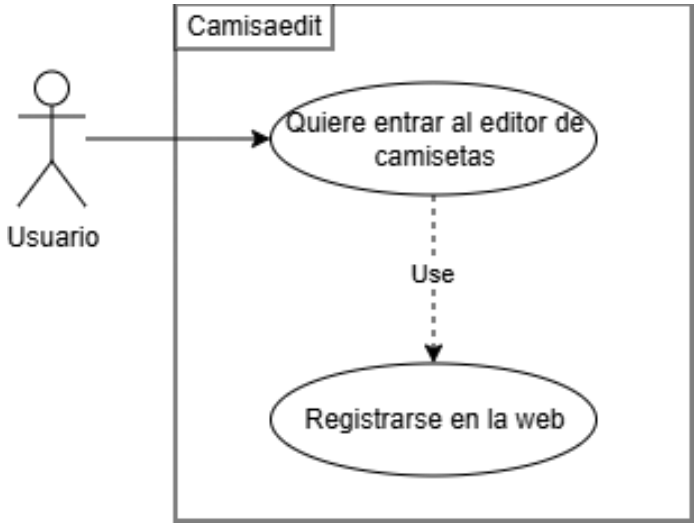
DESCRIPCIÓN

Mini-glosario:

- Los posibles casos de uso.
- Diagramas del diseño del sistema, flujo y de la base de datos
- Explicación detallada de las distintas páginas que forman la aplicación (junto a imágenes de su interfaz)

CASOS DE USO

CASO 1: REGISTRO



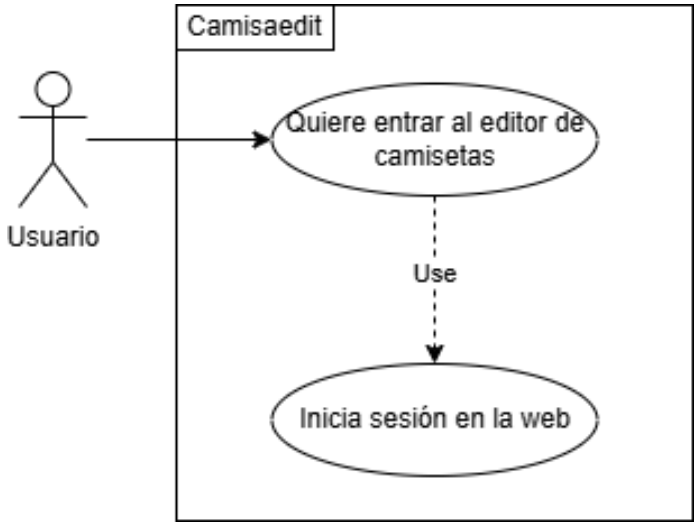
Caso 1: Registro

DESCRIPCIÓN: Registro	
PRECONDICIONES: Usuario no estaba registrado previamente Conexión con la BD	**POSTCONDICIONES**: Usuario Creado Listo para el Inicio de Sesión
DATOS ENTRADA	**DATOS SALIDA**

<table>
<tr><td>Nombre Completo del usuario
Correo del usuario
Teléfono del usuario
Contraseña del usuario (por encriptar)
Contraseña repetida (para cortejar con la primera)</td><td>Correo de la base de datos para chequeo de no duplicados
Nombre Completo del usuario
Correo del usuario
Teléfono del usuario
Contraseña del usuario (por encriptar)</td></tr>
<tr><td>TABLAS:
users</td><td>CLASES:
register.php
userController.php
bdcon.php</td></tr>
<tr><td colspan="2">INTERFACES:
Login.php
register.php
loginReg.js
index.html</td></tr>
<tr><td>FLUJO PRINCIPAL:
1. El usuario accede a la pantalla de registro por vía login.php.
2. Introduce nombre, correo y contraseña.
3. El sistema valida los datos.
4. El sistema almacena la información.
5. Se muestra un mensaje de registro correcto.</td><td>FLUJO ALTERNATIVO:
• 3A. El correo ya existe:
a. El sistema muestra un mensaje de error.
• 3B. Errores de validación de datos (formatos erróneos, contraseñas no encajan…):
a. El sistema muestra un mensaje de error</td></tr>
</table>

Tabla 1: Caso de uso de Registro

CASO 2: INCIO DE SESIÓN



Caso 2: Inicio de sesión

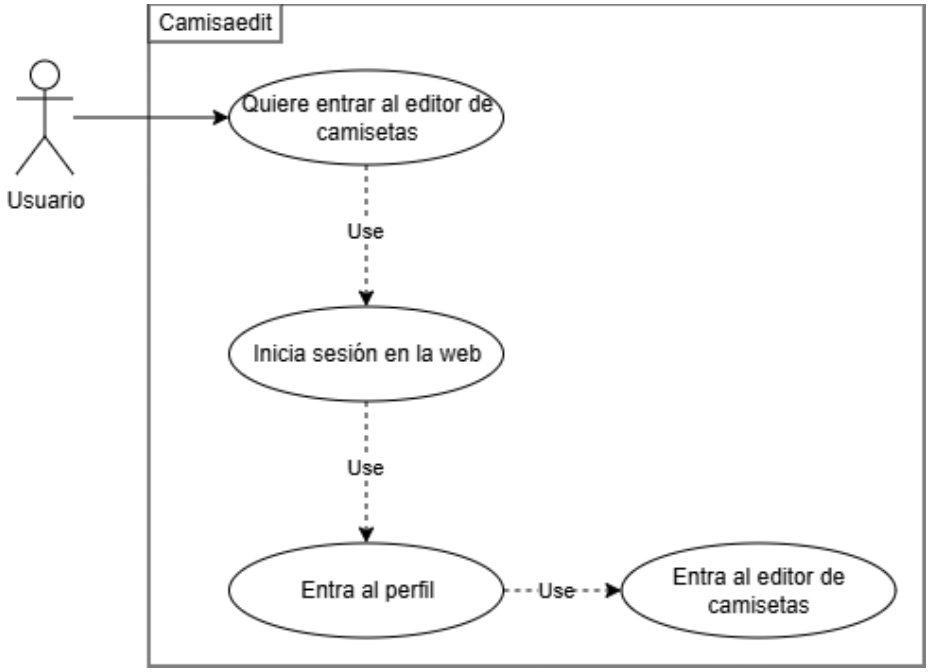
DESCRIPCIÓN: Inicio de sesión	
PRECONDICIONES: Usuario registrado previamente Conexión con la BD	**POSTCONDICIONES**: Sesión del usuario iniciada Listo para entrar al Perfil
DATOS ENTRADA Correo electrónico del usuario Contraseña del usuario (por encriptar)	**DATOS SALIDA** Correo electrónico del usuario (BD) Contraseña del usuario (BD)
TABLAS: users	**CLASES**: login.php userController.php bdcon.php
INTERFACES: login.php loginReg.js	

index.html	

FLUJO PRINCIPAL:
1. Usuario entra a la pantalla de inicio de sesión a través del index.html
2. Usuario escribe sus credenciales
3. El sistema comprueba sus credenciales
4. Se le concede acceso y se le redirige a profile.php tras un mensaje de bienvenida y subir sus datos a la sesión de la página.
	FLUJO ALTERNATIVO:
	• **3A**: Correo incorrecto:
	  ○ Se muestra mensaje de error
	• **3B**: Contraseña incorrecta:
	  ○ Se muestra mensaje de error

Tabla 2: Caso de uso de Inicio de Sesión

CASO 3: CREACIÓN DE ROPA (Y PEDIDO)



Caso 3: Creación de ropa

DESCRIPCIÓN: Creación de ropa	
PRECONDICIONES: Usuario ha iniciado sesión Conexión con la BD	**POSTCONDICIONES**: Ropa creada Pedido creado Entrada lista para enseñarse en Profile

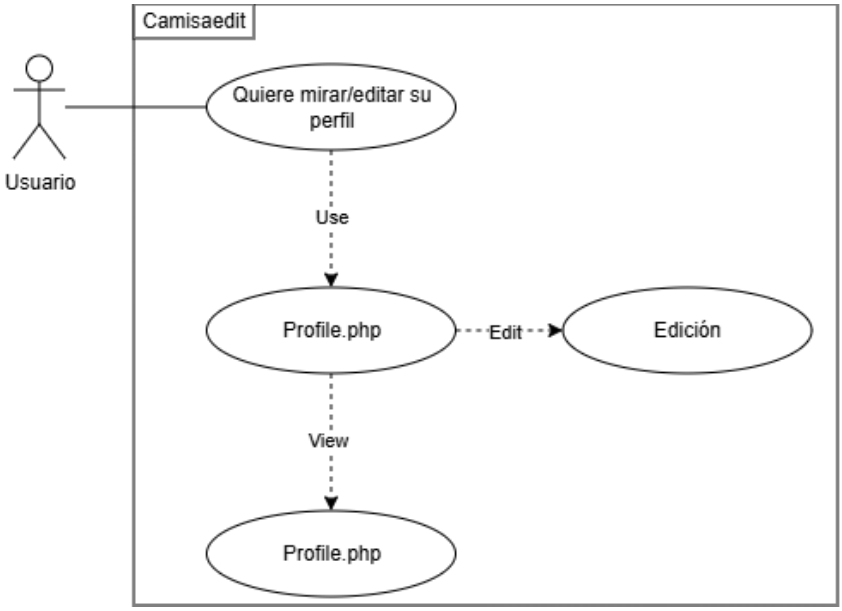
DATOS ENTRADA
• Tamaño
• Color de la base
• Diseño
• Color Diseño
• Contenido de Texto
• Color de Texto
• Posición de Texto
	DATOS SALIDA
	• Ropa
	  ○ Tamaño
	  ○ Color de la base
	  ○ Diseño
	  ○ Color Diseño
	  ○ Contenido de Texto
	  ○ Color de Texto

• Tamaño de texto • Logo • Elevación de logo • Posición de Logo • Tamaño de Logo	○ Posición de Texto ○ Tamaño de texto ○ Logo ○ Elevación de logo ○ Posición de Logo ○ Tamaño de Logo • Pedido ○ Clave Cliente (en la sesión) ○ Clave Ropa ○ Estado de pedido ○ Fecha de registro
TABLAS: Users Pedidos ropa	**CLASES**: editForm.php ropaController.php pedidoController.php bdcon.php
INTERFACES: login.php loginReg.js index.html	
FLUJO PRINCIPAL: **1.** El usuario entra al perfil **2.** El usuario entra al editor de camisetas **3.** El usuario rellena todas las opciones que este quiera para su	**FLUJO ALTERNATIVO:** • **3A**: Validación de datos incorrecta: ○ Se muestra mensaje de error

camiseta personalizada **4.** Una vez esté todo elegido, se avisa al sistema **5.** Se validan todos los campos, y se añade un guión a aquellos que no tengan ningún valor **6.** Se sube a la base de datos **7.** Acto seguido, se crea un nuevo pedido con las claves de tanto el cliente como de la ropa que hemos acabado de mandar **8.** El usuario es devuelto a profile.php	

Tabla 3: Caso de uso de Creación de ropa (y pedido)

CASO 4: VIEW Y EDICIÓN DE PERFIL (por implementar)



Caso 4: Examinación y Edición del perfil (por implementar)

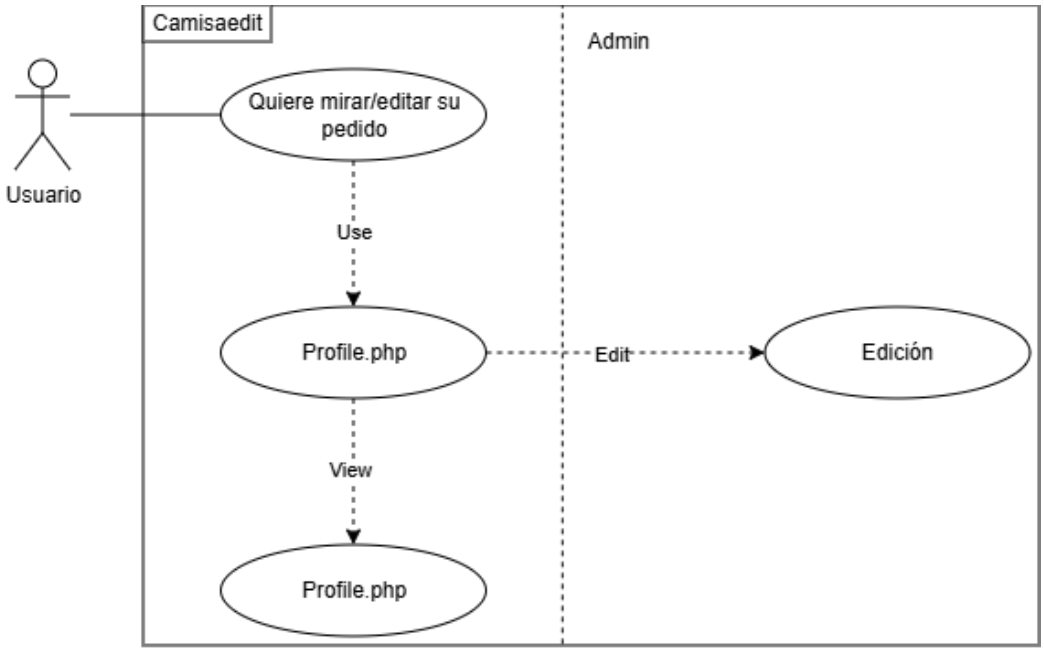
DESCRIPCIÓN: Examinación de Información de usuario	
PRECONDICIONES: Usuario ha iniciado sesión Conexión con la BD	**POSTCONDICIONES**: Información del usuario vista/modificada Listo para entrar al Perfil
DATOS ENTRADA Nombre Completo modificado Correo modificado Teléfono modificado Contraseña de confirmado	**DATOS SALIDA** Nombre Completo modificado Correo modificado Teléfono modificado
TABLAS: users	**CLASES**: Profile.php usersController.php

INTERFACES: Profile.php	

FLUJO PRINCIPAL:
1. Usuario entra a la página del perfil
2. La información del usuario es visible al entrar en la página
3. El usuario pincharía en “editar perfil”
4. Introduce su contraseña para ganar acceso
5. Edita información y la manda, se recarga su página
	FLUJO ALTERNATIVO:
	• **4A**: Contraseña incorrecta:
	  ○ Se muestra mensaje de error
	• **4B**: Validación de datos errónea:
	  ○ Se muestra mensaje de error

Tabla 4: Examinación y Edición del perfil (por implementar)

CASO 5: VIEW Y EDICIÓN DE PEDIDO (Edición Admin-only)



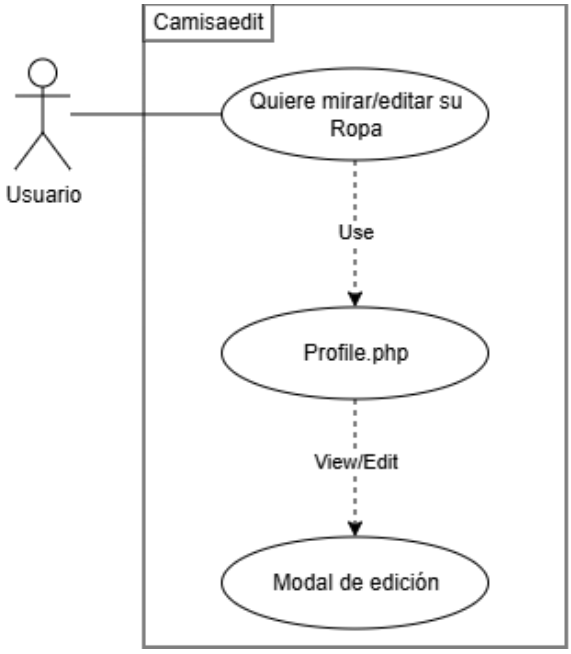
Caso 5: Examinación y Edición del pedido

DESCRIPCIÓN: Examinación de Información del pedido	
PRECONDICIONES: Usuario ha iniciado sesión Existe un pedido asociado al usuario Conexión con la BD	**POSTCONDICIONES**: Información del pedido asociada al usuario vista/modificada Listo para entrar al Perfil
DATOS ENTRADA (Admin-only) Estado de pedido Fecha de inicio Fecha de finalización	**DATOS SALIDA** (Admin-only) Estado de pedido Fecha de inicio Fecha de finalización
TABLAS: pedidos	**CLASES**: Profile.php

	pedidoController.php get_ropa_pedido.php
INTERFACES: Profile.php	
FLUJO PRINCIPAL: **1.** Usuario entra a profile.php **2.** Puede ver la información del pedido en una tabla **3.** Puede ver en más detalle pinchando en “Ver detalles”	**FLUJO ALTERNATIVO:** • **4**: (Admin-only) Edita datos y lo sube a la base de datos • **4A**: Validación de datos errónea: ○ Se muestra mensaje de error

Tabla 5: Examinación y Edición del pedido

CASO 6: VIEW, EDICIÓN Y DELETE DE ROPA



Caso 6: Examinación y Edición de la ropa

DESCRIPCIÓN: Examinación de Información de la ropa	
PRECONDICIONES: Usuario ha iniciado sesión Existe un pedido asociado al usuario Conexión con la BD	**POSTCONDICIONES**: Información de la ropa asociada al usuario vista/modificada Listo para entrar al Perfil
DATOS ENTRADA • Tamaño • Color de la base • Diseño • Color Diseño • Contenido de Texto • Color de Texto • Posición de Texto	**DATOS SALIDA** • Tamaño • Color de la base • Diseño • Color Diseño • Contenido de Texto • Color de Texto • Posición de Texto

• Tamaño de texto • Logo • Elevación de logo • Posición de Logo • Tamaño de Logo	• Tamaño de texto • Logo • Elevación de logo • Posición de Logo • Tamaño de Logo
TABLAS: ropa	**CLASES**: Profile.php ropaController.php get_ropa_details.php
INTERFACES: Profile.php	

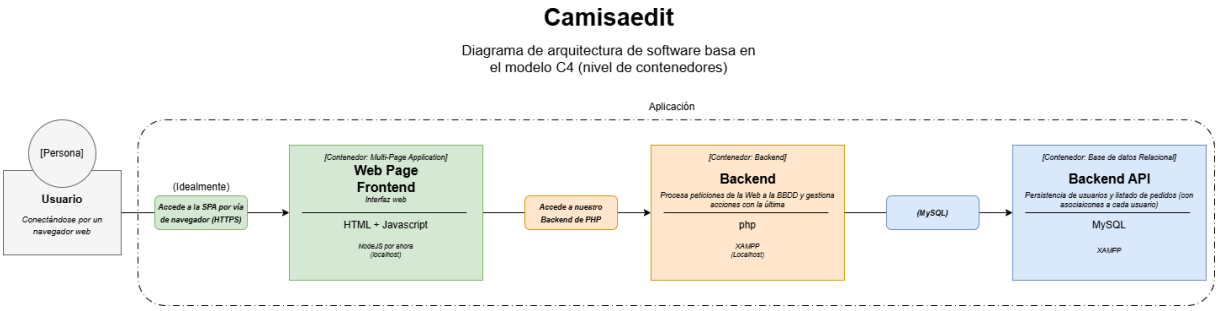
FLUJO PRINCIPAL:
1. Usuario entra a profile.php
2. Puede ver la información del pedido en una tabla
3. Puede ver en más detalle pinchando en “Ver detalles”
4. Editar datos y subida a la base de datos
	FLUJO ALTERNATIVO:
	• **4A**: Validación de datos errónea:
	  ○ Se muestra mensaje de error
	• **4B**: Borrar ropa
	  ○ Se muestra un modal de confirmación

Tabla 6: Examinación y Edición del ropa

DISEÑOS

Arquitectura de la aplicación

Aquí podemos ver la arquitectura de solución para Camisaedit:



Hemos ido por un diseño simple utilizando HTML, JS, PHP y MySQL (XAMPP) para crear una aplicación funcional y completa.

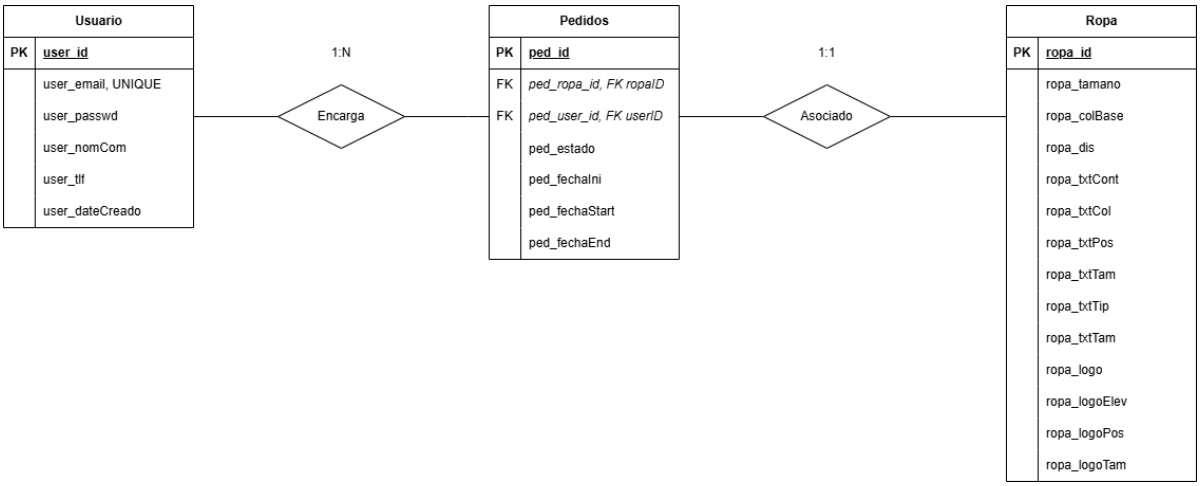


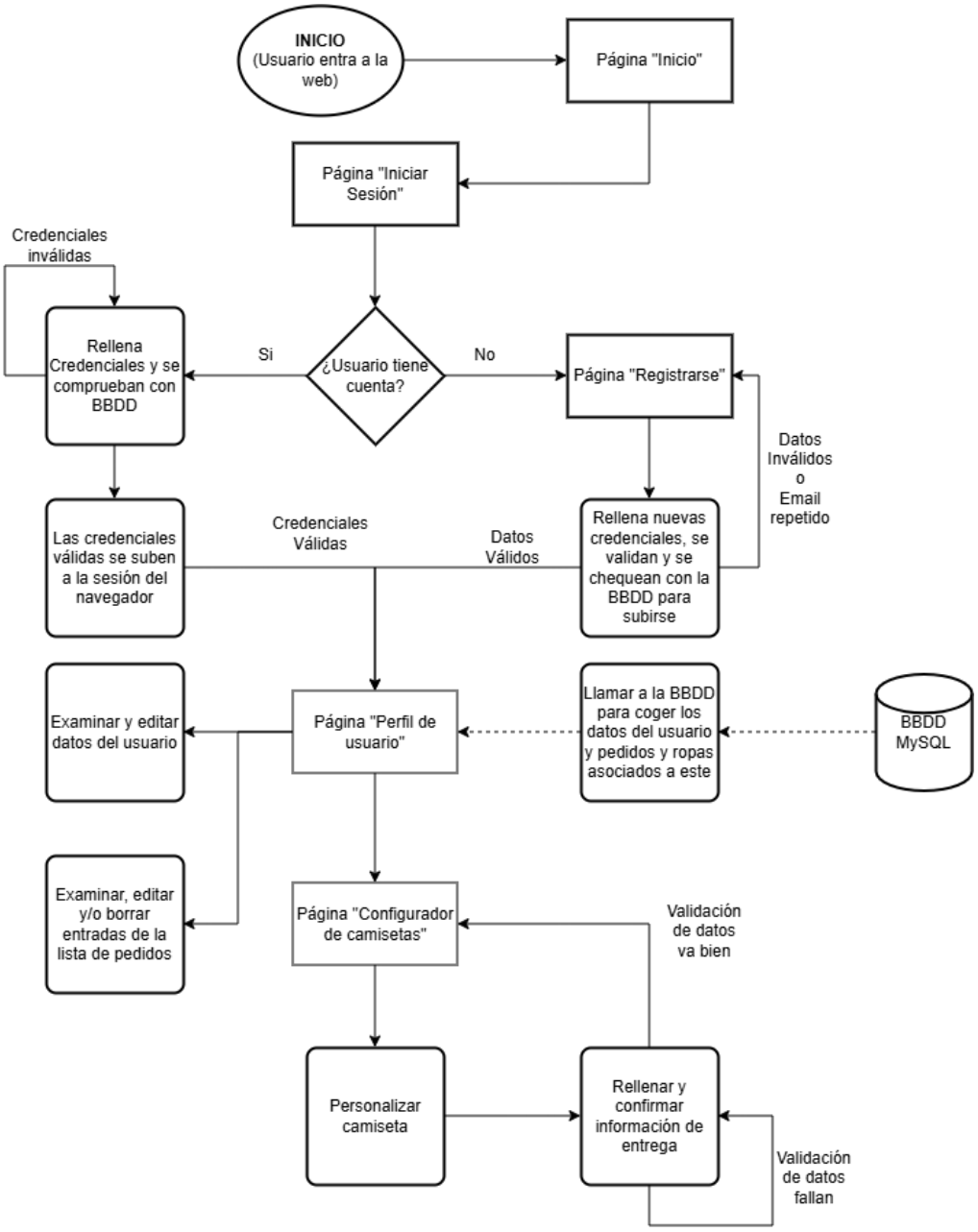
Diagrama de la base de datos.

Aquí vemos el diagrama de toda la base de datos. En efecto, Usuario y Ropa deberían ser una simple relación (1,N) (uno para muchos), pero

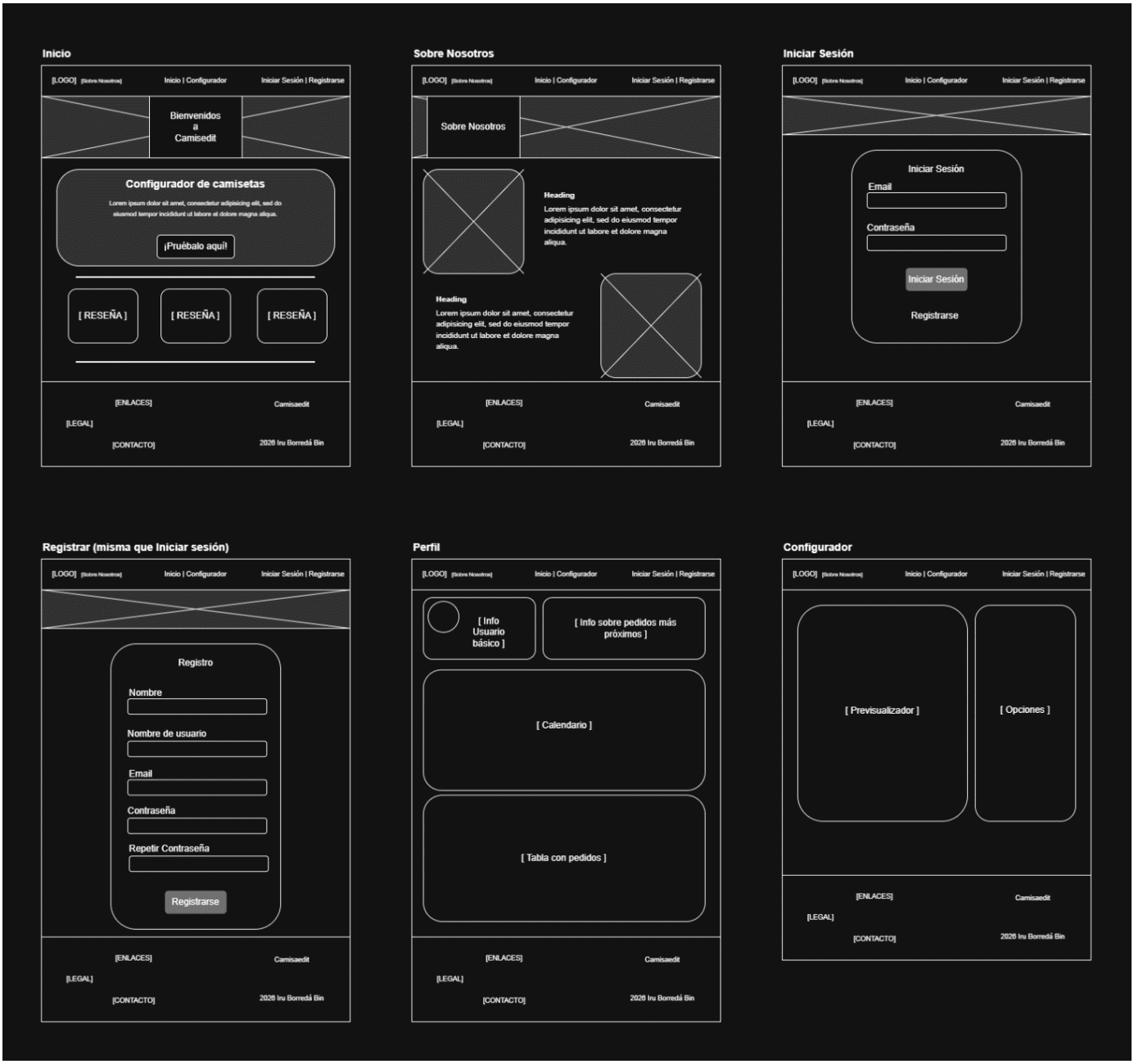
debido a la cantidad ya grande de datos que hay en Ropa, decidí crear la tabla pedidos para que cuando haya que consultar solo información relevante a ello, sea más sencillo hacerlo.

Además, configuré la base de datos de manera que, si se elimina una ropa de la base de datos, su pedido asociado también se borra.

Diagrama de flujo de navegación



Wireframe

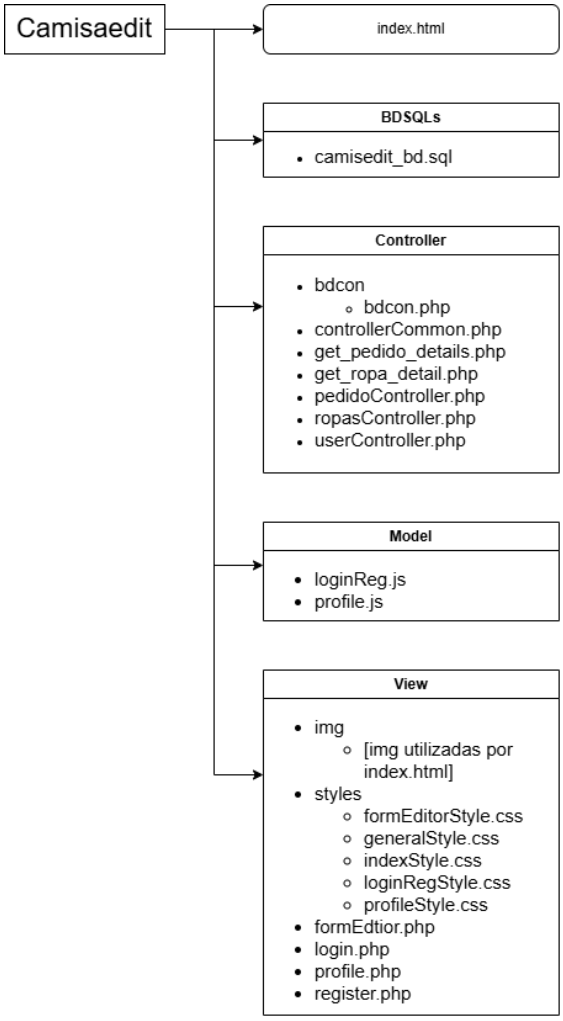


(mejor visto en archivos)

DETALLES DEL PROYECTO

El proyecto ha sido desarrollado siguiendo la arquitectura MVC (Modelo Vista Controlador), donde la “Vista” son las interfaces con las que interactúa el usuario, el Modelo controla las reacciones de la página frente a las acciones del usuario, y Controlador actúa como el puente e intérprete entre la aplicación y el servidor de la base de datos para obtener la información que se pida.

Con eso en mente, el árbol de archivos de nuestro proyecto es el siguiente:



Explicación breve de los archivos:

./Index.html

La página de inicio donde empieza nuestro proyecto. Si en navegando por el directorio pincha en la carpeta del proyecto, se le llevará a esta página primero.

Conecta con: *./view/login.php*

./BDSQLs/camisedit.sql

Aquí se contendrían los archivos necesarios para migrar nuestra base de datos MySQL a otra base de datos MySQL si se necesitara, donde se incluyen instrucciones para recuperar todas las tablas y si se quiere, datos también.

./controller/bdcon/bdcon.php

En este archivo guardamos los datos (y el código) necesarios para conectarnos a la base de datos de MySQL, utilizando MySQLi.

./controller/[…]Controller.php y ./controller/get_[…]_details.com

Los archivos […]Controller.php son donde controlamos tanto la validación de los datos del elemento dentro del servidor como de cómo nos comunicamos con la base de datos desde la aplicación, pudiendo así bajar, borrar, subir y/o editar los datos que se precisen. Aquí también, como en el caso de userController.php, se incluye el subir los datos que querámos preservar en la sesión local.

get_pedido_details.php/get_ropa_detail.php son los archivos dedicados a contener una consulta de tipo AJAX (Asynchronous JavaScript and XML) para poder cambiar el modal de edición de datos de un pedido/camiseta cambiar de forma dinámica sin tener que pelearnos con php para que pueda comunicar con el front de esta manera, mandando así un JSON con todos los datos relevantes al frontend donde profile.js lo recibe.

./model/...

En estos archivos controlamos dos cosas en su mayoría:

- La interactividad de los distintos elementos de la página, incluyendo qué enseñan (como un modal de confirmación), como en algunos casos redirecciones a otra página
- Como se insertan los datos de forma dinámica donde deberían, y también validaciones dentro del cliente antes de mandar cualquier formulario

./view/...

Aquí se guardan todas las interfaces (con la excepción de index.html), todas ellas con la extensión php puesto que es necesario debido a que todas las páginas requieren de conexión con la base de datos para funcionar correctamente y mostrar los datos relevantes al usuario conectado.

Explicación de los sistemas de la aplicación:

Sistema de inicio de sesión y registro

El sistema de registro e inicio de sesión de Camisaedit está compuesto de cinco archivos distintos:

- ./view/login.php y ./view/register.php que contienen los formularios respectivos a cada proceso.
- ./model/loginReg.js que controla la reactividad de las dos páginas y valida todos los datos de forma dinámica dentro del cliente.
- ./controller/userController.php que controla que todos los datos estén bien validados antes de ser subidos, al igual que controlaría cualquier otra gestión que se tuviera que hacer entre la aplicación y la tabla “users” en la base de datos.

Registro

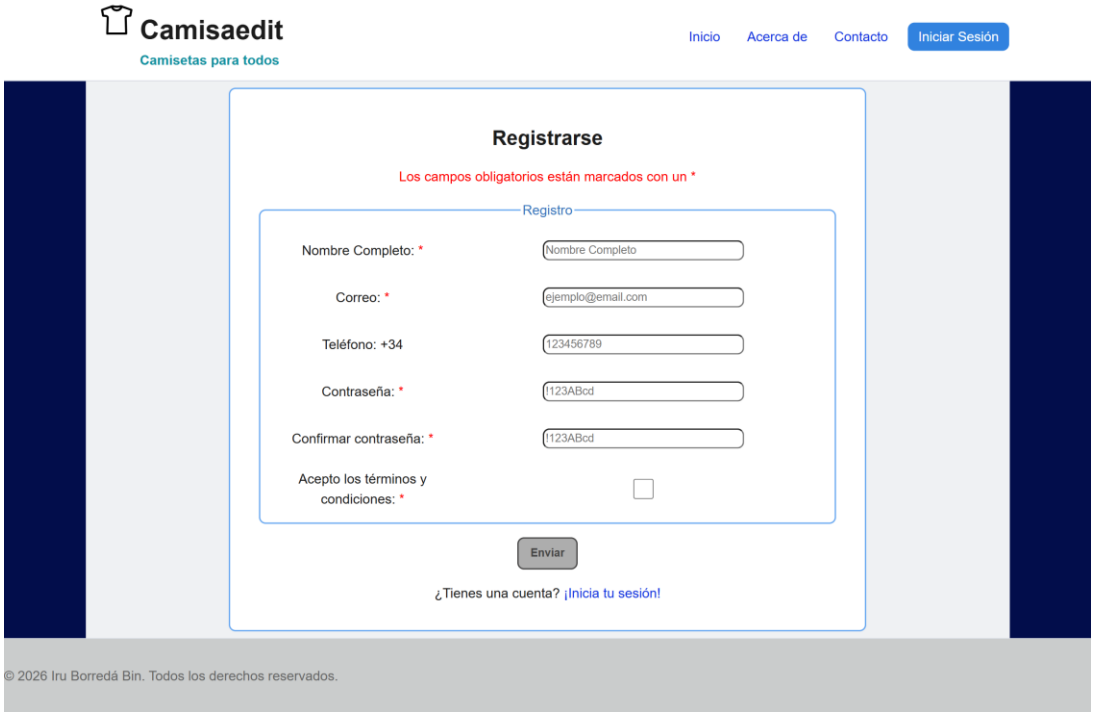


Ilustración a: Interfaz de registro.php

La primera parte de todo empieza en la página de registro, a la que el usuario podrá acceder desde la página de login.php si no tiene una cuenta propia (link proporcionado).

El formulario que se ve recoge los siguientes datos:

- Nombre Completo (**obligatorio**).
- Correo electrónico (**obligatorio**).
- Teléfono con el prefijo español “+34”.
- Contraseña (**obligatoria**).
- Confirmar contraseña (**obligatoria**).
- Una casilla de verificación para aceptar los términos y condiciones que controla que el botón de “Enviar” esté habilitado.

Lo validamos tanto en el lado cliente como en el lado servidor para prevenir ataques de inyección SQL. Las validaciones que tienen que aprobar cada campo *son (loginReg.js y líneas 7 a 49 de userController.php)* :

- Primero se asegura que ningún campo obligatorio esté vacío.
- El Nombre, el Correo y la Contraseña son validados con el uso de un patrón regex.
    - Aparte, dentro del servidor nos aseguramos que el email que pone el usuario no esté repetido en la base de datos, saltando un error en caso de haberlo. *(userController.php líneas 52 a 59)*
    - Las reglas de la contraseña siendo: un mínimo y máximo de entre 8 a 15 caracteres, en los cuales debe haber al menos una letra maýuscula, una minúscula y un carácter especial
- El campo teléfono (que no es obligatorio) se valida asegurándose que hay nueve dígitos en la entrada sin ningún carácter string en medio.
    - También, nos aseguramos que dentro del propio campo teléfono no se pueda poner ningún otro carácter que no sea un número, monitorizando los inputs del teclado (controlamos cuando el usuario quiere borrar el campo también)
- La confirmación de la contraseña se comprueba simplemente asegurándonos que el valor del campo original es el mismo.
- Finalmente, nos aseguramos de que el usuario ha leído y aceptado los términos de servicio antes de habilitar el botón de “enviar”

Adicionalmente, en la capa cliente de esta página, controlamos las validaciones en tiempo real cada vez que hay un nuevo input del teclado en los campos, igual que en login.php.

Tras verificar que todos los datos son correctos, se estos se suben a la base de datos, y si todo está correcto, se recibe un aviso de éxito y el usuario es redirigido a login.php directamente.

Inicio de sesión

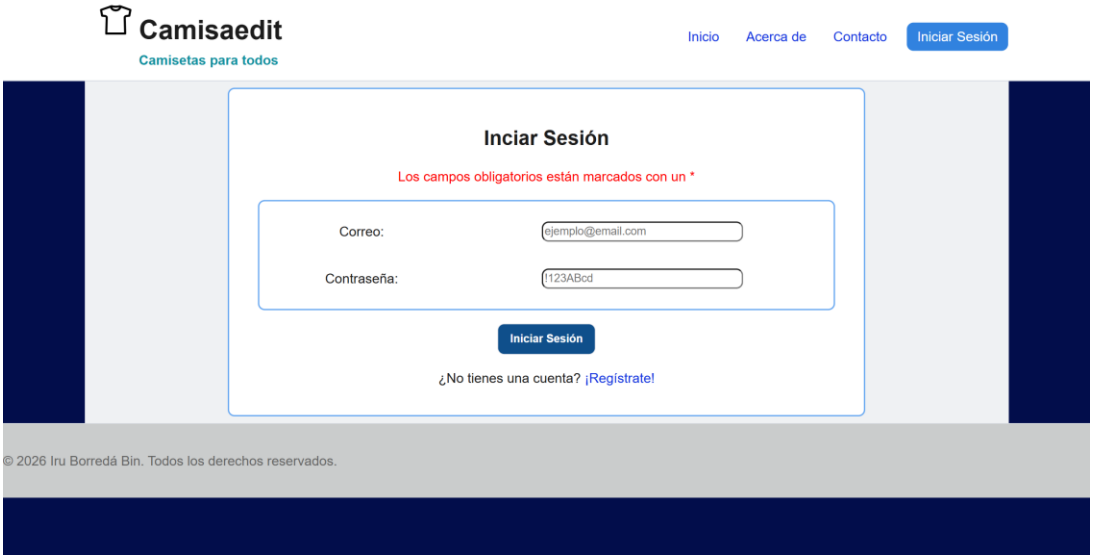


Ilustración b: Interfaz Inicio de Sesión

Aquí tenemos la página de inicio de sesión, que se puede acceder directamente desde el index.html, register.php y profile.php.

Este formulario es más simple, y está hecho para solo para iniciar la sesión del usuario. Para este propósito, solo pedimos dos campos aquí:

- El Correo electrónico con el que se han registrado (**obligatorio**)
- La Contraseña asociada a ese correo (**obligatorio**)

Igual que con el registro, validamos que los datos estén correctos antes de ser mandados *(loginReg.js y líneas 81 a 106 de userController.js)*:

- El Correo y la Contraseña son validados con el uso de un patrón Regex.

- Al haber puesto el constraint “unique” en el correo de nuestra base de datos, podemos utilizarlo como un buen dato para buscar la información del usuario en cuestión y traer los datos.

Una vez se hayan validado las credenciales, la página lanza una alerta dando la bienvenida al usuario, y tras guardar los datos del usuario en una sesión *(líneas 108 a 135 de userController.php)*, redirige a este a directamente a profile.php.

Sistema de creación de camisetas y pedidos

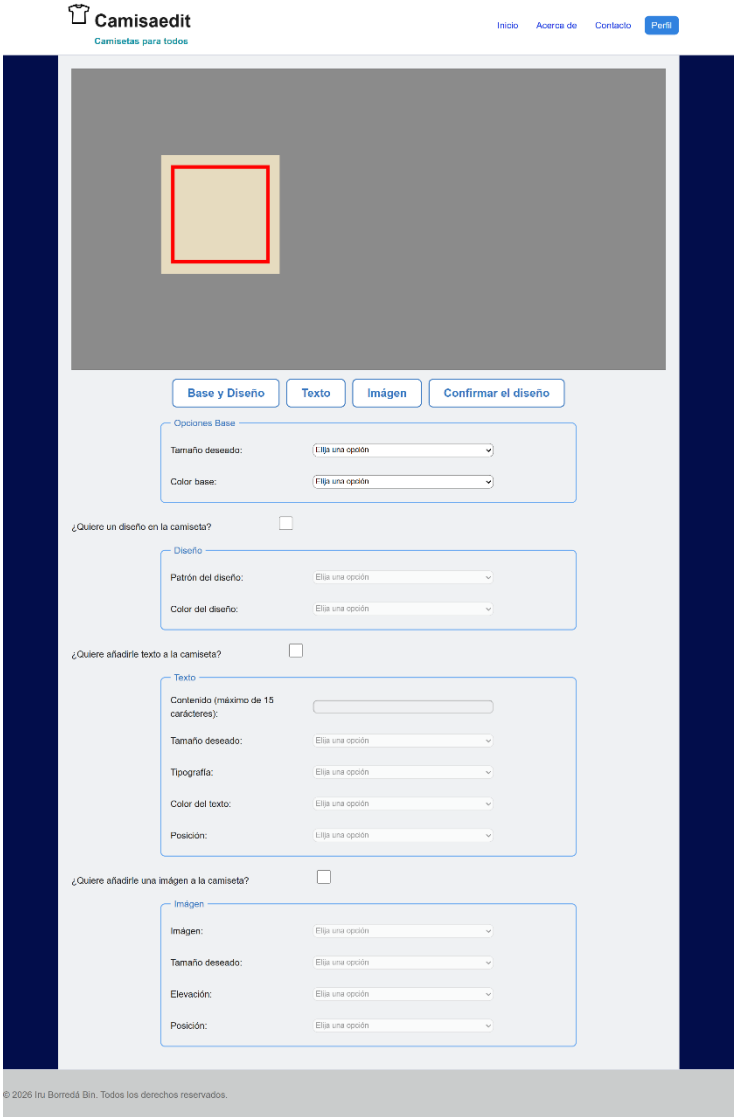


Ilustración c: Interfaz de Editor de Camisetas

Antes de ver cómo funciona el profile.php (que es más extenso y dónde converge todo), miramos rápidamente el editor de camisetas de formEditor.php, que solo contiene exclusivamente un formulario de opciones para personalización de camisetas.

En este sistema entran en juego los siguientes archivos:

- ./view/formEditor.php que contiene el formulario
- ./controller/ropasController.php para el CRUD con la tabla “ropa” de la base de datos
- ./controller/pedidosController.php para el CRUD con la tabla “pedido”, a la que la tabla “ropa” está enlazada.
- ./controller/bdcon/bdcon contiene las credenciales y el código para conectar con la base de datos

Aquí tenemos un formulario de selects simple en el cual se controla qué opciones adicionales se quieren en la camiseta con el uso de checkboxes. Estos más tarde controlan que de no haberse habilitado una de las opciones, los campos vacíos se cambien a tener solo un guión “-“ para así no estar nulos dentro de la base de datos.

Ahora tan solo se verifica que todos los campos estén llenados (incluyendo los obligatorios del tamaño y el color base) y se suben a la base de datos y se manda un aviso de vuelta indicando al usuario del éxito *(líneas 5 a 14 de ropasController.php).*

Acto seguido, formEditor.php también llama a pedidoControlller.php para crear un pedido asociado a la ropa que hemos acabado de crear, cogiendo el código de esta y del usuario en sesión para subir los datos al igual que la fecha de creación del pedido.

Problema encontrado y solucionado:

Aquí tuve un problema con el botón que necesitaba utilizar para que los datos de la sesión del usuario se pasaran de profile.php a formEditor.php, debido a que cuando se pasa de una página a otra utilizando cosas del front, se pierden los datos de la sesión.

Hice un arreglo rápido montando un mini-formulario alrededor del botón para cambiar de un lado a otro que esperaba a que el botón fuese pulsado. *(Líneas 239 a 251 de profile.php)*

```
<?php //Para que así podamos acceder a el editor con nuestra sesión de usuario
if (!empty($_GET['act'])) {
    echo '<script type="text/javascript">window.location.href="formEditor.php";</script>';
    exit();
} else {
    ?>
    <form action="profile.php" method="get">
        <input type="hidden" name="act" value="run">
        <button class="btn" id="btn_create_new_camiseta" type="submit">Crear Nueva Camiseta</button>
    </form>
    <?php
}
?>
```

Ilustración d: Líneas 239 a 251 de profile.php

Sistemas del Perfil

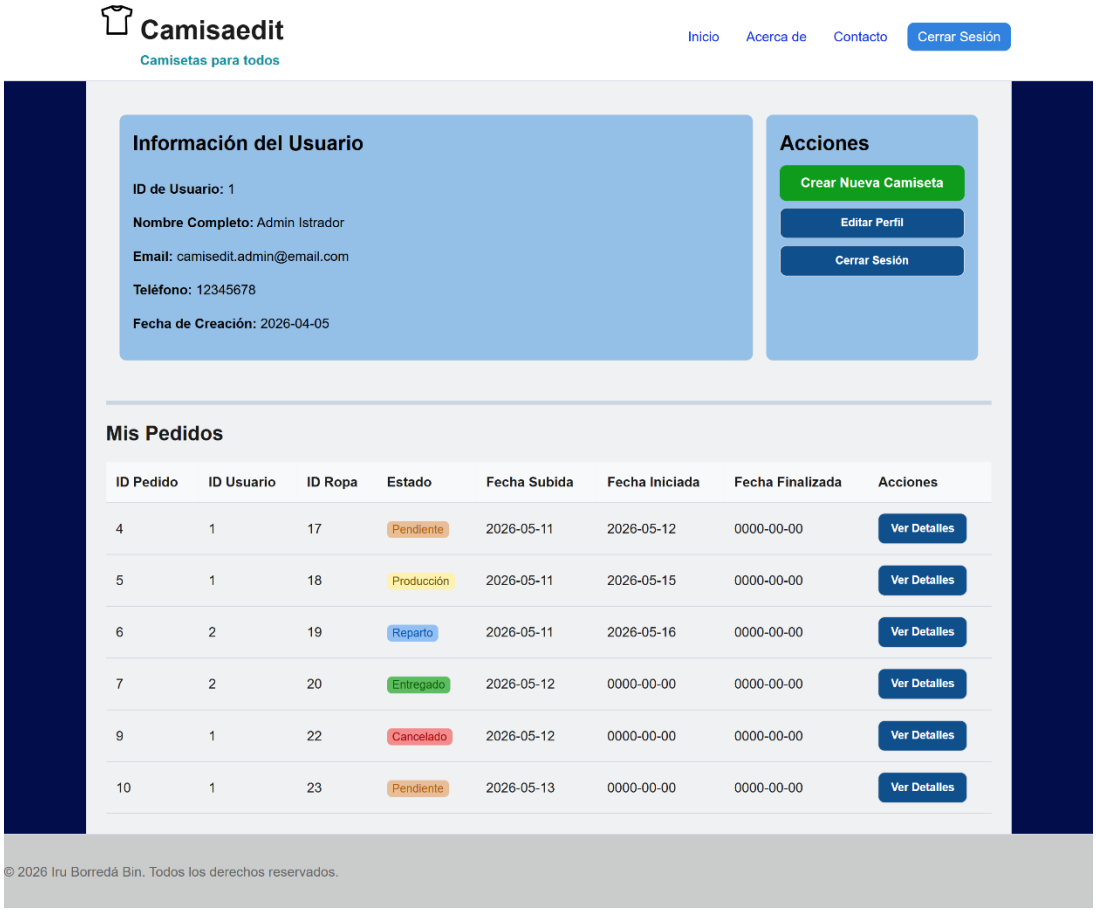


Ilustración e: Interfaz de profile.php

En esta página convergen las diferentes partes de nuestra aplicación, la parte del usuario, la de la ropa y la del pedido. Los archivos que están involucrados en esta página son:

- ./view/profile.php que contiene tanto los sitios para mirar la información como los modales con formularios que se cambian de forma dinámica según lo que se quiera editar
- ./model/profile.php que contiene el código para los componentes reactivos de la página, al igual que la tabla de pedidos (admin puede ver todo) y de recoger objetos JSON mandados para rellenar los modales

- ./controller/ropasController contiene el CRUD y las validaciones necesarias de un ítem de la tabla “ropa” y get_ropa_details.php ayuda a pasar esos datos del servidor a un objeto JSON, que el frontend sí que puede utilizar.
- ./controller/pedidoController contiene el CRUD y las validaciones necesarias de un ítem de la tabla “pedidos” y get_pedido_details.php ayuda a pasar esos datos del servidor a un objeto JSON, que el frontend sí que puede utilizar.
- ./controller/bdcon/bdcon contiene las credenciales y el código para conectar con la base de datos

Debido a todas las partes que hay en solo esta página, lo voy desglosar todo en las siguientes partes:

- Información del Usuario
- Renderizado de la tabla de pedidos
- Cómo recibe el modal los datos
- Edición de los datos del modal
- Eliminación de una Ropa y Pedido

Información del usuario

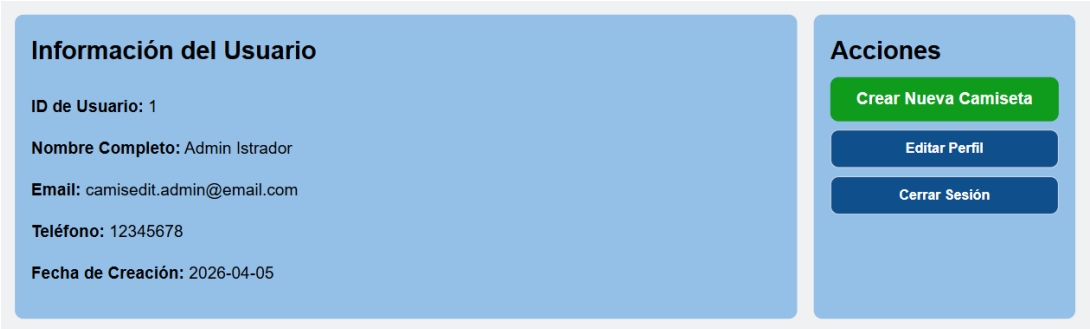


Ilustración f: Sección de información del perfil

Gracias a que nos hemos subido los datos del usuario a la sesión, podemos simplemente pegarlos donde deseemos dentro la página sin necesidad de llamar a la tabla “users”, teniendo los datos a mano.

En el momento en el que estoy escribiendo esta memoria, no se pueden editar los datos del usuario, pero al ya tener como editar tanto los pedidos como las ropas, no será difícil implementarlo, al igual que si se decidiría borrar la cuenta.

Renderizado de la tabla de pedidos

Mis Pedidos

ID Pedido	ID Usuario	ID Ropa	Estado	Fecha Subida	Fecha Iniciada	Fecha Finalizada	Acciones
4	1	17	Pendiente	2026-05-11	2026-05-12	0000-00-00	Ver Detalles
5	1	18	Producción	2026-05-11	2026-05-15	0000-00-00	Ver Detalles
6	2	19	Reparto	2026-05-11	2026-05-16	0000-00-00	Ver Detalles
7	2	20	Entregado	2026-05-12	0000-00-00	0000-00-00	Ver Detalles
9	1	22	Cancelado	2026-05-12	0000-00-00	0000-00-00	Ver Detalles
10	1	23	Pendiente	2026-05-13	0000-00-00	0000-00-00	Ver Detalles

Ilustración g: Sección de la tabla de pedidos (Admin)

La tabla la formamos utilizando los datos que recibimos de la base de datos (que cambian según el usuario sea solo un cliente o el propio administrador), y directamente utilizando php e inyectándolo en el html. *(Líneas 21 a 35 y 40 a 57 de profile.php respectivamente).*

```
// Hacemos una consulta que guardamos para traer a nuestra página de perfil los pedidos del usuario logueado.
// Para esto, utilizamos el ID del usuario que tenemos guardado en la sesión.
if (isset($_SESSION['user_id']) && $_SESSION['user_id'] == 1) {
    $userID = $_SESSION['user_id'];
    $pedidos = consult_getPedidosAdmin($conn);
    console_log('Todos los pedidos (Vista Administrador):');
    console_log($pedidos);
    $tableOutput = generatePedidosTable($pedidos);
} else if (isset($_SESSION['user_id'])) {
    $userID = $_SESSION['user_id'];
    $pedidos = consult_getPedidosByUserID($conn, $userID);
    console_log('Pedidos obtenidos para el usuario con ID ' . $userID . ':');
    console_log($pedidos);
    $tableOutput = generatePedidosTable($pedidos);
} else {
    console_log('No se encontró un ID de usuario en la sesión. No se pueden cargar los pedidos.');
}
```

Ilustración h: Líneas 21 a 35 de profile.php

```php
// Con esta variable guardaremos el código HTML generado para mostrar la tabla de pedidos,
// que luego se inyectará en el HTML del perfil.
$tableOutput;
2 references
function generatePedidosTable($pedidos)
{
    $row = "";
    foreach ($pedidos as $pedido) {
        $row .= "<tr>";
        $row .= "<td>" . $pedido['ped_id'] . "</td>";
        $row .= "<td>" . $pedido['ped_user_id'] . "</td>";
        $row .= "<td>" . $pedido['ped_ropa_id'] . "</td>";
        $row .= "<td><span class='tag tag-" . $pedido['ped_estado'] . "'>" . $pedido['ped_estado'] . "</span></td>";
        $row .= "<td>" . $pedido['ped_dateIni'] . "</td>";
        $row .= "<td>" . $pedido['ped_dateStart'] . "</td>";
        $row .= "<td>" . $pedido['ped_dateEnd'] . "</td>";
        $row .= "<td><button class='btn' id='btn_ver_pedido' data-ped-id='" . $pedido['ped_id'] . "' data-ropa-id='" . $pedido['ped_ropa_id'] . "'>Ver Detalles</button></td>";
        $row .= "</tr>";
    }
    return $row;
}
```

Ilustración i: Líneas 40 a 57 de profile.php

El botón “Ver detalles” nos abre un modal cuyos datos mostrados son diferentes en función de qué pedido queramos examinar, editar o borrar.

Cómo recibe el modal los datos

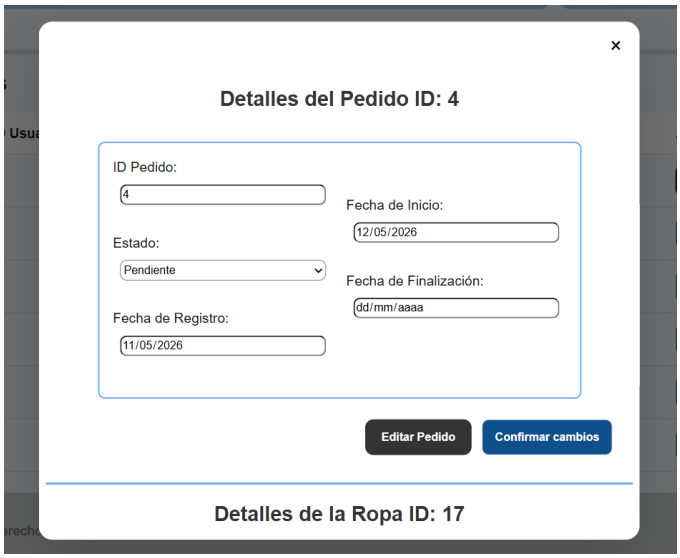


Ilustración j: Pantallazo del modal

Aquí entran en acción los archivos get_pedido_details.php y get_ropa_details.php, que utilizan la técnica AJAX para devolver objetos JSON de las consultas que le hagamos a la base de datos en vez de tener que hacerlo todo por php como para renderizar la tabla. *(Líneas 21 a 36 de profile.js)*

```
try {
    // Cargar detalles del pedido
    const responsePedido = await fetch(`../controller/get_pedido_details.php?ped_id=${pedidoID}`);
    const dataPedido = await responsePedido.json();

    if (dataPedido.error) {
        alert(dataPedido.error);
        return;
    }

    // Cargar detalles de la ropa
    const responseRopa = await fetch(`../controller/get_ropa_details.php?ropa_id=${ropaID}`);
    const dataRopa = await responseRopa.json();

    if (dataRopa.error) {
        alert(dataRopa.error);
        return;
    }
```

Ilustración k: Líneas 21 a 36 de profile.js

Una vez se han recogido los datos en formato JSON de forma satisfactoria, cogemos el objeto y ponemos sus valores en los campos de los formularios del modal. Llamamos esta secuencia de acciones cada vez que pinchamos en el botón “Ver Pedido”, que contiene como atributos tanto la id del usuario que creó el pedido, como el id de la ropa que tiene asociada, de la cual se pueden sacar todos los datos.

Edición de los datos del modal

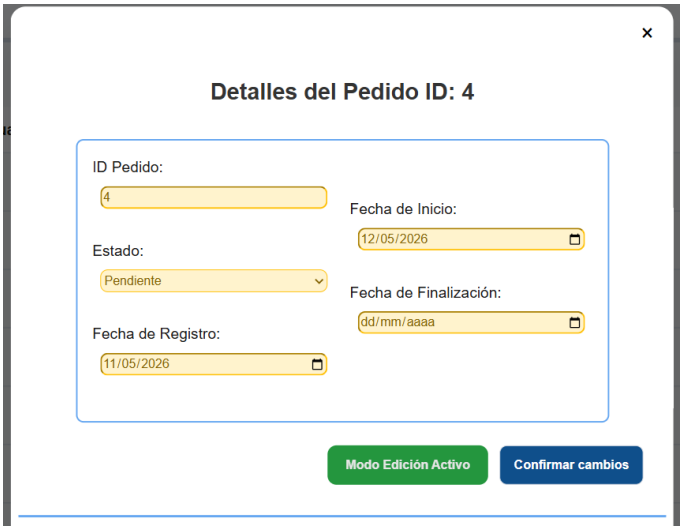


Ilustración I: Modal de edición con el modo de edición activo

Tanto para la parte del pedido como la de la camiseta, todos los campos permanecen como readonlys hasta que se pulase el botón de “Editar […]”, tras lo cual el usuario podrá cambiar cualquier dato al que tenga acceso (salvo por las IDs) y luego enviarlo para la base de datos tras validarse todo. Las dos partes van separadas una de la otra.

También, mientras que los usuarios pueden editar todos los campos camiseta, solo aquellos que sean administradores pueden editar aspectos de los pedidos.

NOTA: Para las fechas de Inicio y Finalización, he utilizado Javascript para que automáticamente no se permita editar las camisetas a una fecha más antigua que la del día actual.

Eliminación de una Ropa y Pedido

Ilustración m: Modal de confirmación de borrar

Para eliminar un pedido y una ropa, es un proceso “automático” gracias a los constraints que hay entre la tabla “ropa” y la tabla “pedido”, en la que la borrarse una ropa causa que el pedido asociado se borre también (CASCADE ON DELETE).

Por lo tanto, basta con solo borrar la camiseta en cuestión para que no tengamos que aparte quitar el pedido de la tabla.

Una vez se llega al final del modal, podemos hacer click a “Eliminar Camiseta” para que nos salga un segundo modal que nos pida confirmar nuestra decisión, y si accedemos a borrarlo, nos devolverá a profil.php, recargando la página

TECNOLOGÍAS

Las tecnologías y herramientas utilizadas para este proyecto.

Lenguajes utilizados	
JS	**Javascript** **Descripción**: Es un lenguaje de programa ligero e interpretado que es fundamental en el desarrollo web para crear contenido dinámico e interactivo. **Uso**: Es una parte crucial de nuestro proyecto, ya que se ocupa directamente de manejar la reactividad de la web frente a las acciones del usuario.
HTML	**HTML**. **Descripción:** Es un lenguaje de etiquetas que es el código estándar fundamental utilizado para estructurar y desplegar páginas web por vía de etiquetas. **Uso**: Estructura de toda la web, desde la página de inicio hasta el perfil.
CSS	**CSS**. **Descripción:** Es un lenguaje de diseño utilizado para dar una apariencia visual personalizada a los documentos HTML en la web. **Uso**: Hace la interfaz con la que los usuarios interactuarán atractiva y visualmente más accesible.

	PHP. **Descripción:** Es un lenguaje de programación diseñado específicamente para el desarrollo web del lado del servidor. **Uso**: Nos permite establecer y gestionar una conexión con nuestra base de datos, incluyendo cualquier interactuación con esta y mandar datos al front-end para mostrarlos.

Base de datos	
	MySQL **Descripción:** Es sistema de gestión de bases de datos relacionales **de** código abierto**,** basado en el lenguaje SQL**.** Es una de las tecnologías más populares para almacenar, organizar y recuperar datos de manera eficiente en aplicaciones web y de software. **Uso:** Se utiliza para guardar y hacer persistir toda la información y los datos que se recojan desde nuestra aplicación web, además de permitirnos llamarlos más tarde.

Entornos de desarrollo	
	XAMPP **Descripción:** Es un paquete de software libre y de código abierto que permite instalar de forma sencilla un servidor local (localhost) en tu ordenador, incluyendo en sí tanto un intérprete de PHP, Apache y MySQL preconfigurados. **Uso:** Usamos sus funciones de intérprete de PHP y para utilizar la base de datos PhPMyAdmin en local sin necesidad de montar un sistema en la web.

	Visual Studio Code **Descripción:** Es un entorno de desarrollo ligero de código abierto muy popular para el desarrollo web debido su flexibilidad y modularidad de sus extensiones. **Uso:** Edición de código, validador de sintaxis, visualizador de archivos del proyecto… Prácticamente el único IDE que se ha utilizado.

Control de versiones	
	Git / Github **Descripción:** • Git es un sistema de control de versiones distribuido que rastrea cambios en el código localmente, permitiendo gestionar el historial de proyectos. • GitHub es una plataforma en la nube que aloja repositorios de Git, facilitando la colaboración, el trabajo en equipo y el intercambio de código **Uso:**

Diagramas y diseños	
	Draw.io **Descripción:** Es una aplicación gratuita, de código abierto y basada en la web para crear diagramas profesionales, diagramas de flujo, mapas conceptuales, organigramas y esquemas de red. **Uso:** Ha sido utilizado para prácticamente todos los diagrama que están en la memoria.

METODOLOGÍA

Metodología usada y justificación

Debido a estar estudiando al mismo tiempo que haciendo las prácticas y otros eventos en mi vida que han ocupado mucho de mi tiempo, no he podido adherirme a ninguna metodología como tal. Me gustaría imaginar que debido a mi manera de trabajar – iterando sobre un problema una y otra vez hasta lograr solucionarlo –, y luego hacer un “sprint” intensivo, se adhiere más a la metodología AGILE, pero no me arriesgaría en decir que lo he seguido totalmente.

En cuanto a horas, sobre-estimé el tiempo que podía echarle a este proyecto y también cuanto tiempo tardaría en entender más sobre rutas y controladores:

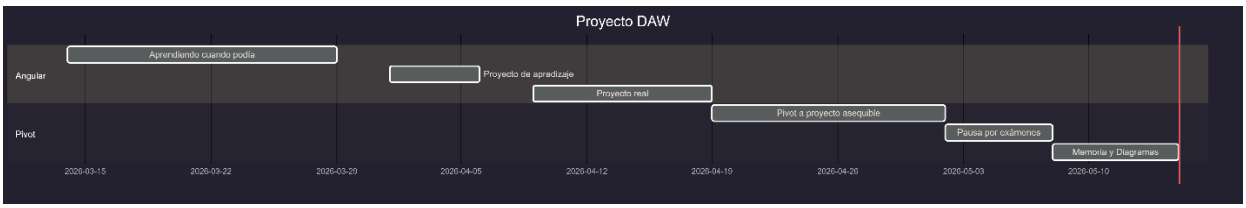
Horas Estimadas (Previas a implementación)

- Angular : 20 horas totales
    - Aprendizaje: 15 horas
    - Proyecto de prueba: 5 horas
- R02F01 – Sistema de Inicio de Sesión y Registro : 4 horas
- R02F02 – El Usuario debe poder ver datos de su cuenta y una lista de pedidos que haya realizado: 2 horas
- R02F02T01 - R02F02T03 – Edición de datos del pedido: 3 horas
- R03 – El programa ha de tener un formulario de colección de opciones elegidas para la camiseta: 2 hora
- **HORAS TOTALES: 31 Horas**

Horas Estimadas (Posterior a implementación)

- Angular: 25 horas totales (aprox)
  - Aprendizaje: 15 horas
  - Proyecto de prueba: 5 horas
  - Entender cómo funciona mi proyecto después de empezar con Angular: 5 horas
- Posterior al Pivot (Cambio a todo Vanilla): 58 horas (aprox)
  - Análisis de los requisitos y objetivos prioritarios: 8 horas
  - Diseño de la arquitectura: 2 horas
  - Diseño del Backend: 4 horas
  - Diseño de Interfaces/Frontend: 6 horas
  - Desarrollo del Frontend: 13 horas
  - Desarrollo del Backend: 14 horas
  - Documentación: 11 horas
- **HORAS TOTALES: 83 Horas**

Diagrama de Gantt.



(Aparece en mejor calidad en los archivos)

PRESUPUESTO

Datos simulados utilizando los pagos actuales para programadores en prácticas en 2026 (De 6 € a 12 €).

Tarea	Horas	€/Hora	Coste (€)
Aprendizaje de Angular	25 (aprox)	6 €	150 €
Análisis de requisitos y objetivos	8	12 €	96 €
Diseño de la Arquitectura	2	12 €	24 €
Diseño del Backend	4	12 €	48 €
Diseño del Frontend	6	12 €	72 €
Desarrollo Frontend	13	12 €	156 €
Desarrollo Backend	14	12 €	168 €
Documentación	11	12 €	132 €
TOTALES	**83 Horas**		**846 €**

README y GIT.

Habrá dos READMEs, uno en la carpeta general del repositorio, el otro dentro de la propia carpeta del proyecto.

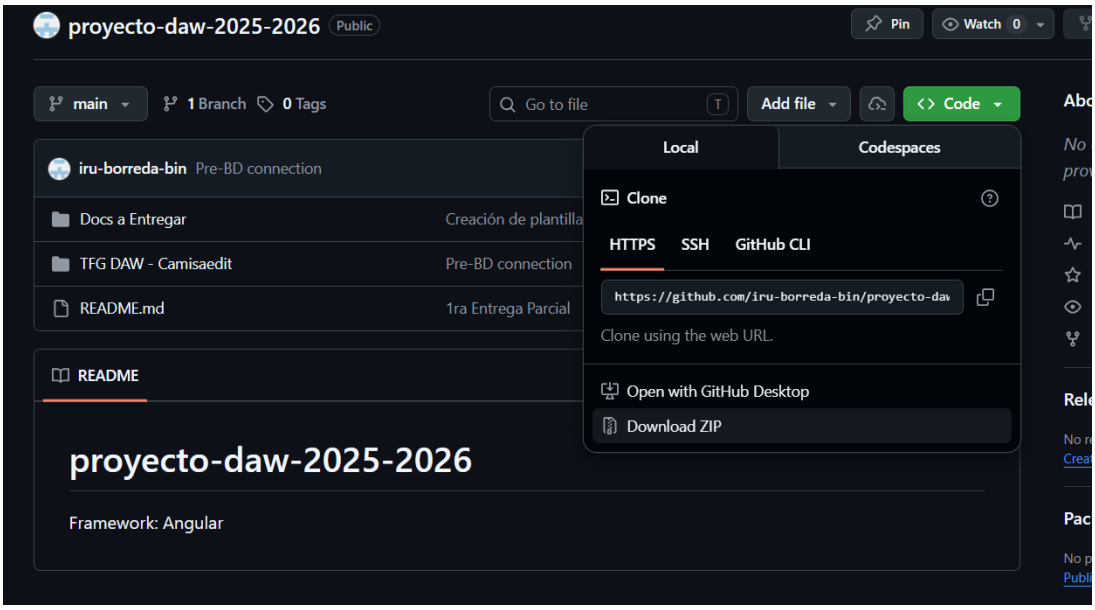
https://github.com/iru-borreda-bin/proyecto-daw-2025-2026

INSTALACIÓN Y DESPLIEGUE

Instalación

Esta guía de instalación asume que usted ya tiene instalado XAMPP en su máquina con Apache y MySQL pre-configurados.

- **Paso 1**: Descárguese el proyecto desde Github (Haga click en "<> Code" y busque la opción "Descagar ZIP" en las opciones del desplegable como aparece señalado en el pantallazo.



- **Paso 2**: Localice dónde tiene la carpeta de XAMPP instalada (normalmente en C: si no le ha dicho al Wizard otra dirección). Por defecto está en "C:\xampp"
- **Paso 3**: Entre en la carpet, busque la carpeta htdocs y entre en ella. (estamos en "C:\xampp\htdocs")
- **Paso 4**: Una vez ahí, creé una carpeta que se llame "phpProjects", dentro de esta carpeta podremos ejecutar nuestro proyecto, ya que

una vez que encendamos el Apache de XAMPP, nuestro navegador podrá interpretarlo.

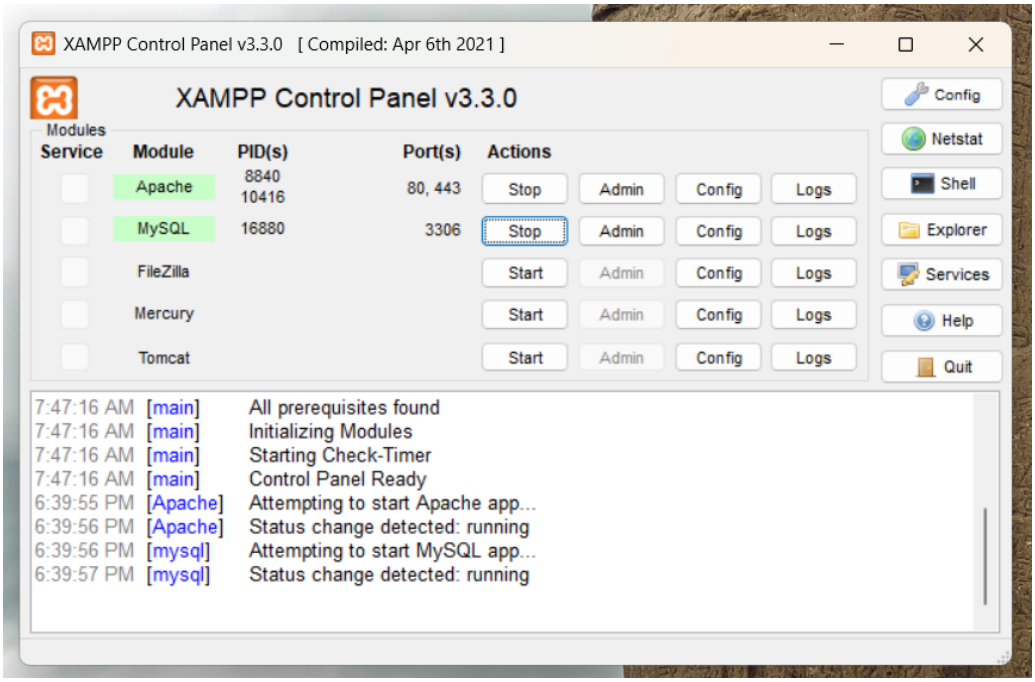
- **Paso 5**: Una vez haya creado “phpProjects” y se meta, descomprima el zip aquí.

El proyecto al final de este proceso tendría que estar en la siguiente dirección:

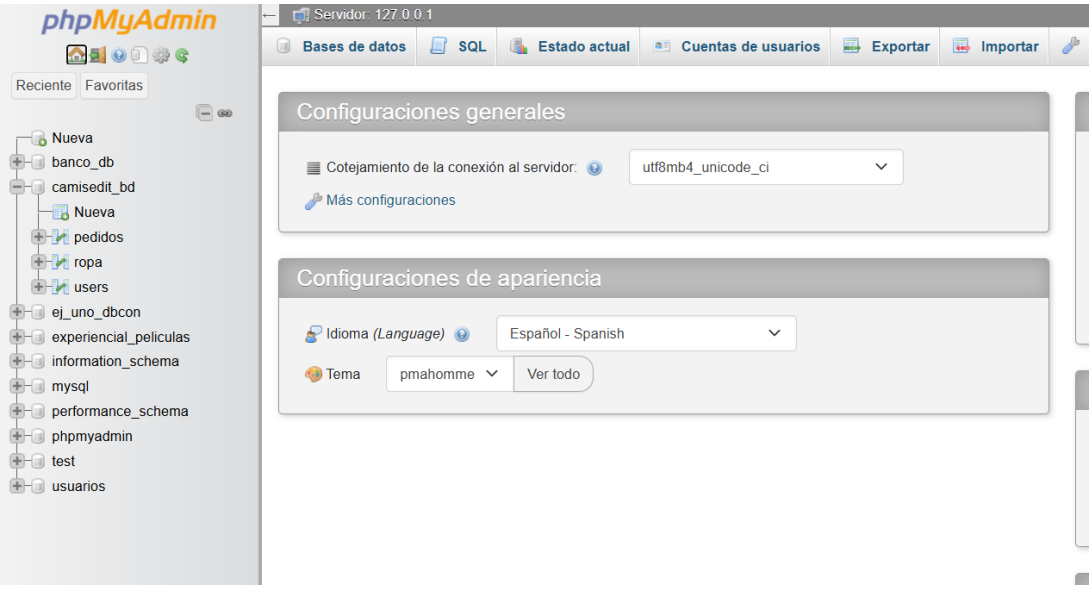
“C:\xampp\htdocs\phpProjects\proyecto-daw-2025-2026”

Tras esto, una vez que se tiene el proyecto descomprimido, localize el archivo “camisedit_bd.sql” dentro de la carpeta “BDSQLs”, ya que la va a necesitar para realizar una importación de base de datos.

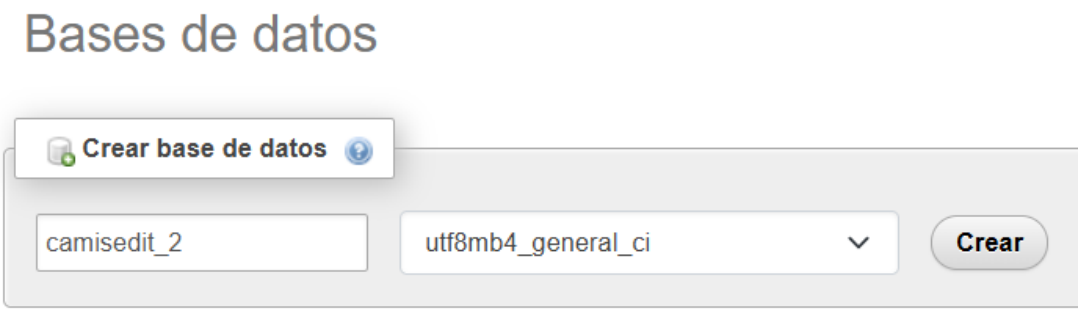
- **Paso 1**: Abra XAMPP y encienda Apache y luego MySQL tras esperar a que el nombre de Apache se vuelva verde.



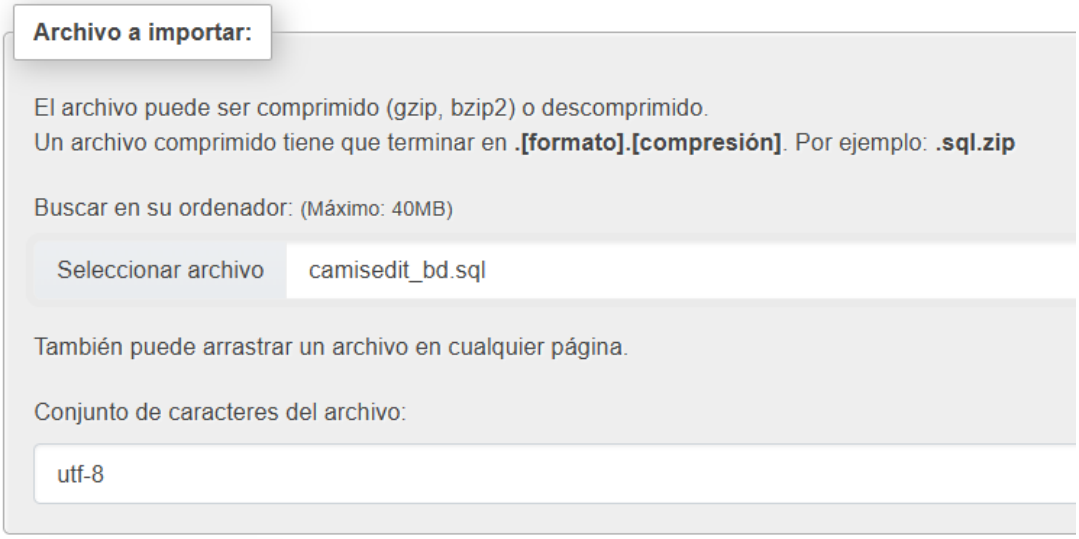
- **Paso 2:** Coja su navegador preferido y escriba en la barra de búsqueda http://localhost/phpmyadmin y pulse enter para entrar a la base de datos.



- **Paso 3:** Una vez hecho esto, creamos una nueva base datos con el nombre "camisedit_bd" pulsando el enlace "Nueva" en la barra lateral. (Tengo que utilizar camisedit_2 debido a que ya tengo un "camisedit_bd")

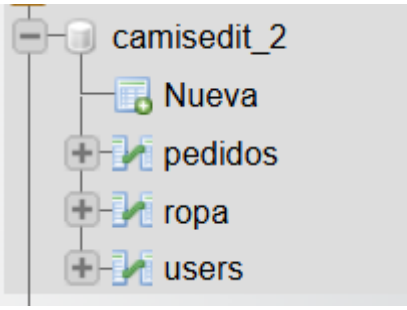


- **Paso 4:** Una vez pinche en “Crear”, será redirigido a la página de la base datos. Ahí encontrará una barra superior donde tiene que entrar a “importar”. Ahí interactúe con la primera opción “Archivo a Importar” y seleccione nuestro archivo “camisedit_bd.sql”.



De ahí vaya abajo del todo y pulse en “Importar”.

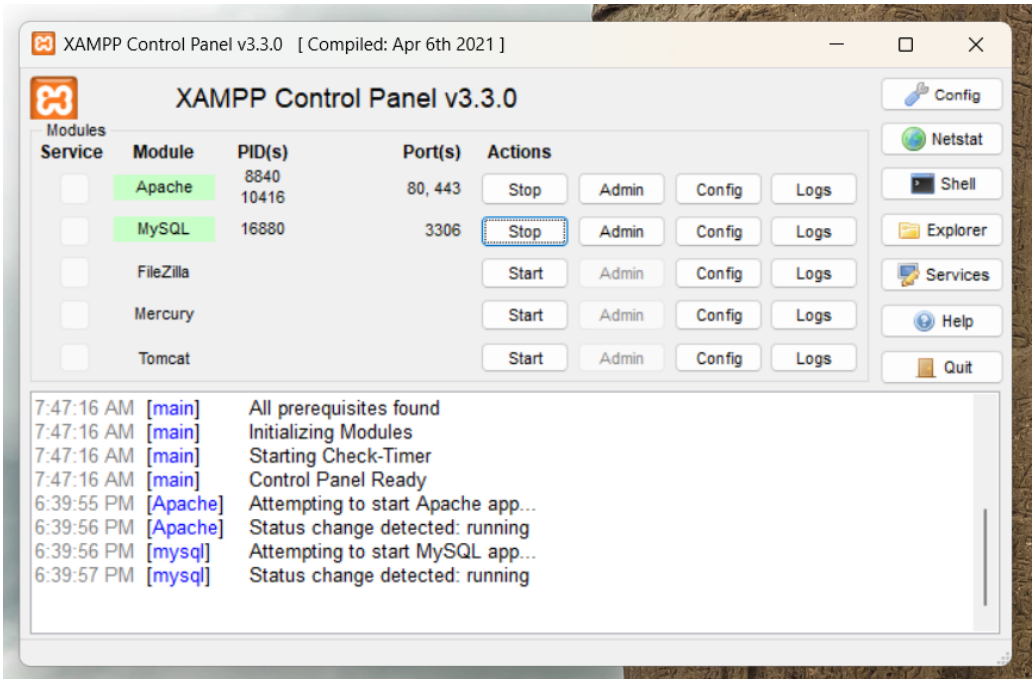
Una vez hecho todo esto, ya se le debería creado la base de datos en su servidor:



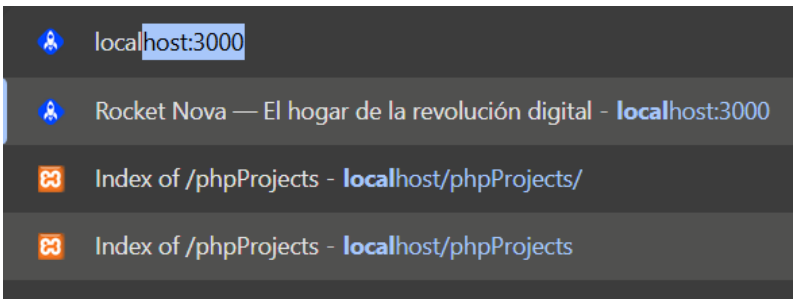
Despliegue

Una vez que todo está listo:

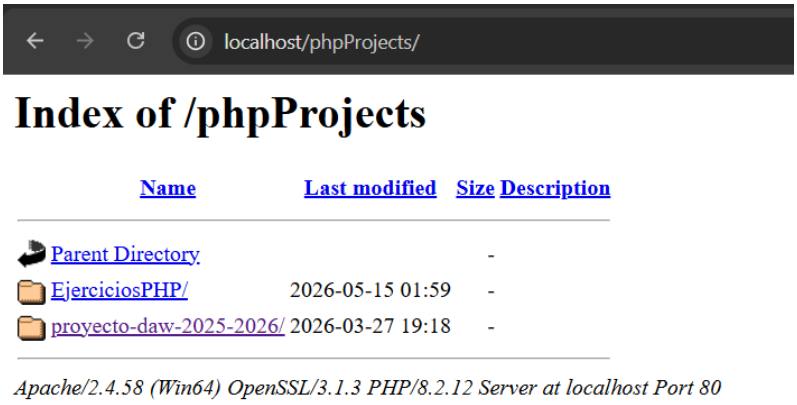
- **Paso 1**: Abra XAMPP y encienda Apache y luego MySQL tras esperar a que el nombre de Apache se vuelva verde.



- **Paso 2**: Una vez tanto el servidor de Apache como el de MySQL estén levantados, elija su navegador de preferencia, y en el buscador de enlaces, busque “localhost/phpProjects”, donde le aparecerá un enlace a “Index of phpProjects”, haga click sobre el link.



- Y una vez esté ahí, se encontrará con la siguiente pantalla (ignore la carpeta EjerciciosPHP):



  Una vez aquí, navegue a “proyecto-daw-2025-2026”, y dentro de ahí, haga click sobre “TFG DAW – Camisaedit”.

Y una vez hecho eso, le saldrá la pantalla principal (“index.html”) y podrá utilizar la aplicación sin problemas.

TRABAJOS FUTUROS

Debido a todo los que me gustaría hacer con este proyecto, listare mis ideas:

- Mejoras con la base de datos:
  - Roles para los usuarios como Administradores y clientes (así también podríamos añadir a varios Administradores con sus propias cuentas.)
  - Constraints de pedido:
    - No se puede editar o borrar el pedido de ninguna forma una vez que la camiseta ya esté en producción (lo cual no creo que sería muy difícil de implementar).
- Mejoras para Perfil:
  - Poder editar y/o borrar el perfil
  - Poner una imagen de perfil
  - Los usuarios solo pueden “Cancelar” un pedido, no borrarlo directamente
  - No se debería poder editar la fecha de registro del pedido (bug todavía no solucionado)
  - En caso de eliminación de perfil, que todos los pedidos sean “Cancelados”.
  - Contraseñas encriptadas utilizando hash.
- Mejoras a el editor de camisetas:
  - Un previsualizador 2D como Armoria como he mencionado previamente
  - Los formularios de opciones estarían contenidos en una barra lateral, accesible a través de *tabs*.
  - Debería poder haber una vista de galería para ver las imágenes y los logos por defecto

    - Se debería poder subir tu propia imagen para la camiseta.
- Mejoras de carácter general: Mejorar este proyecto a una aplicación SPA usando React o Angular, con una conexión a una base de datos Supabase.

CONCLUSIONES

Siendo honesto, no estoy seguro por dónde empezar.

Como cuando terminé DAM, hacer el proyecto de fin de grado ha sido una experiencia tensa a la par que una con la que he aprendido mucho por mi cuenta. He refrescado mi cabeza una vez más en cómo organizarme de una manera efectiva y productiva, y también a pivotar a otro carril siguiendo en la misma dirección si veo que lo que he de afrontar me superaba.

He puesto en práctica todo lo que he aprendido, si, pero al haber estado estudiando y al mismo tiempo haciendo prácticas, no he podido echarle el tiempo suficiente como para implementar tecnologías que hemos visto en clase de forma rápida y un tanto atropellada que me hubiera gustado trabajar aquí, como, por ejemplo, utilizar MongoDB para mi proyecto, el cual era el plan original pero para cuando lo estábamos dando en clase ya era demasiado tarde.

Como tal, opino que este proyecto es un buen ejemplo del nivel actual de mis habilidades como un desarrollador web con las situaciones en las que me he encontrado, y espero no hacer más que mejorar desde aquí.

REFERENCIAS

Según las normas APA.
Cada referencia se acompañará de un texto descriptivo con el apartado del proyecto asociado.

Formato:

Autor, A. A. (Año de publicación). Título de la página. Recuperado de URL

W3Schools Documentation (Varios Autores, Año desconocido):

Javascript: https://www.w3schools.com/js/default.asp

SQL: https://www.w3schools.com/sql/default.asp

CSS: https://www.w3schools.com/css/default.asp

PHP: https://www.w3schools.com/php/default.asp